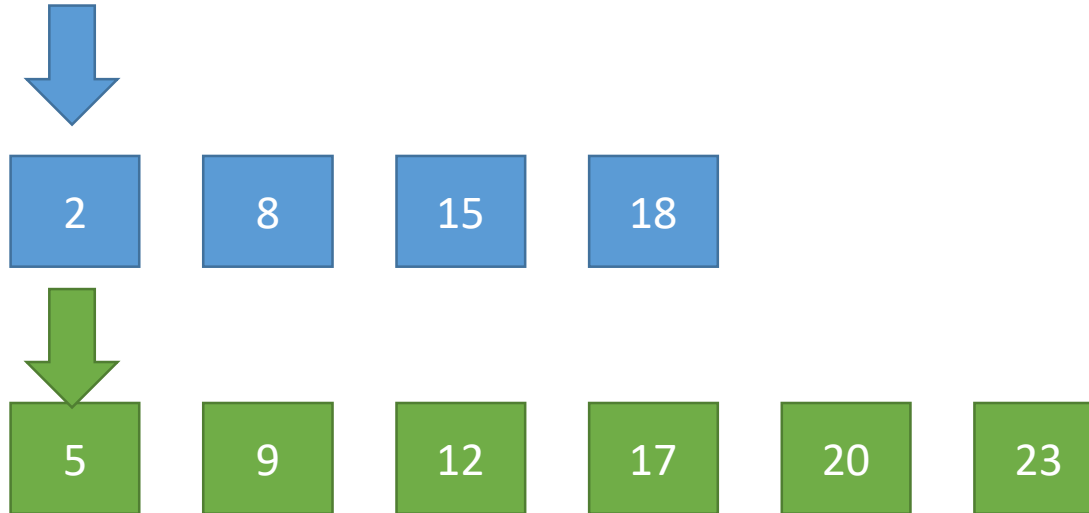


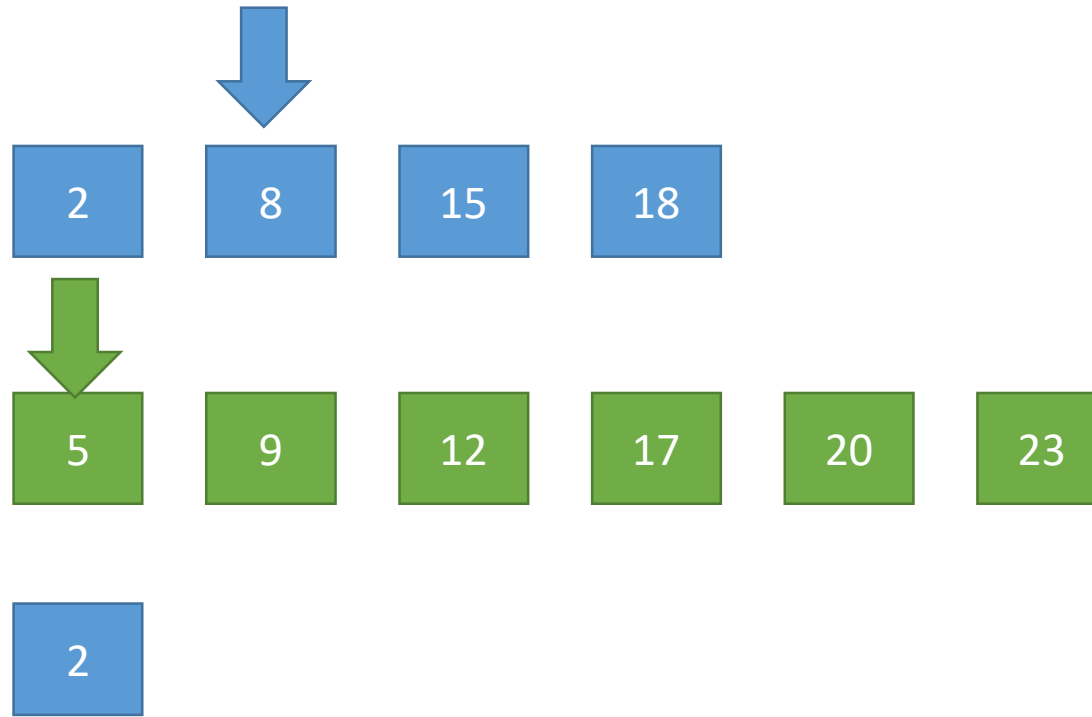


Merge Sort

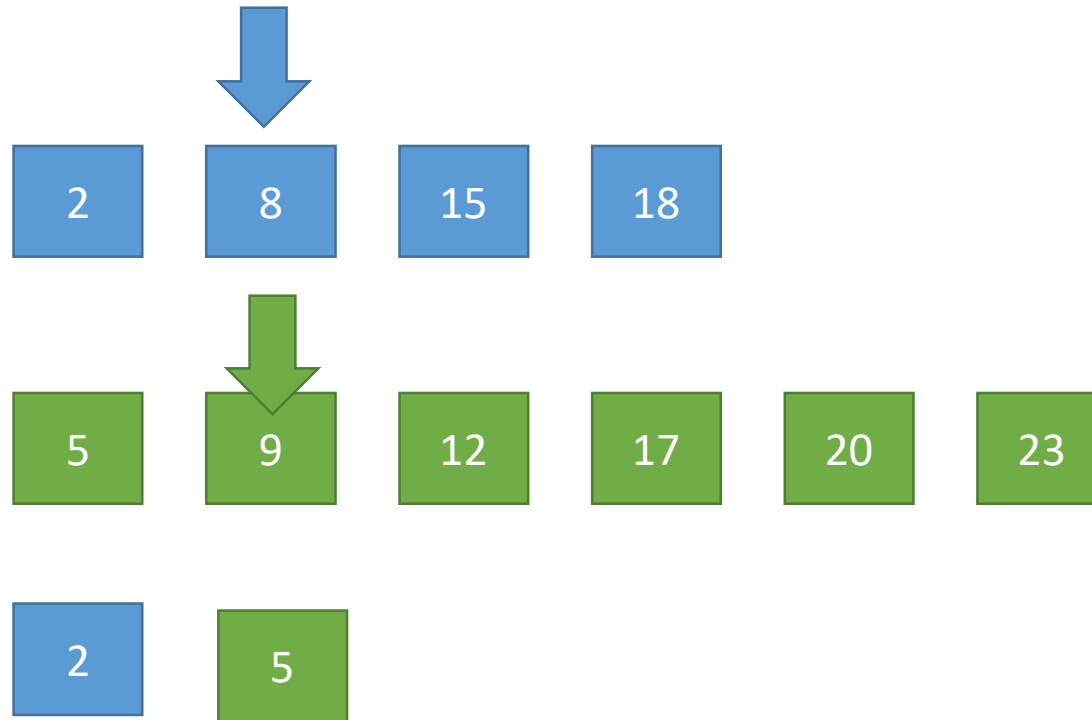
Merging Two Decks of Cards



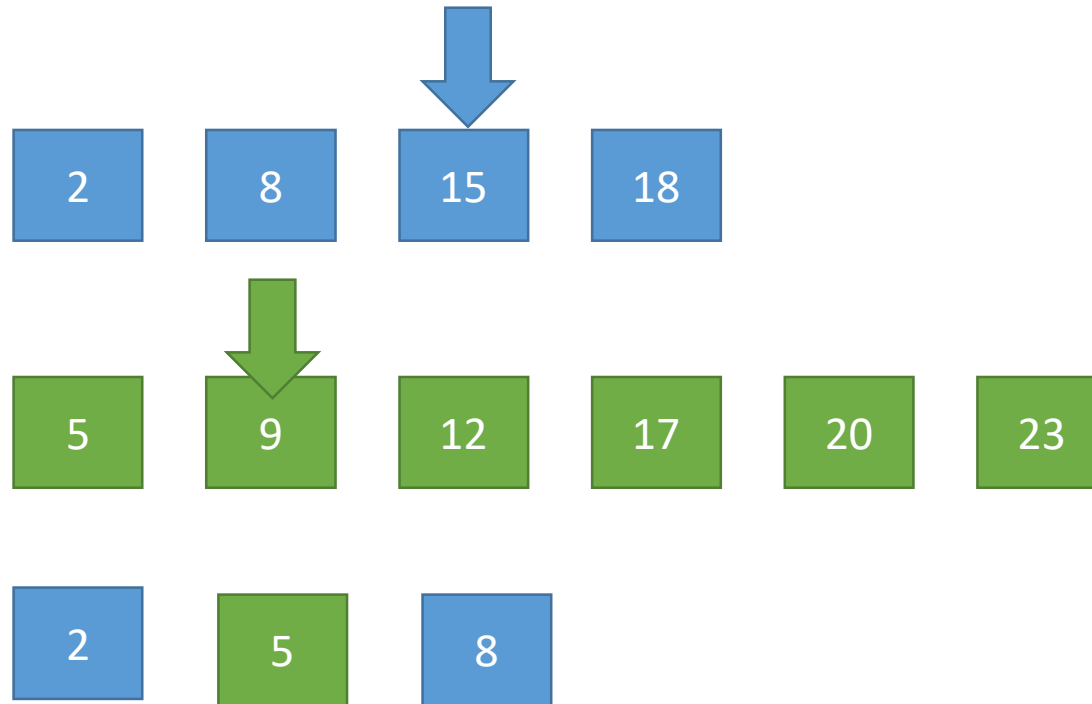
Merging Two Decks of Cards



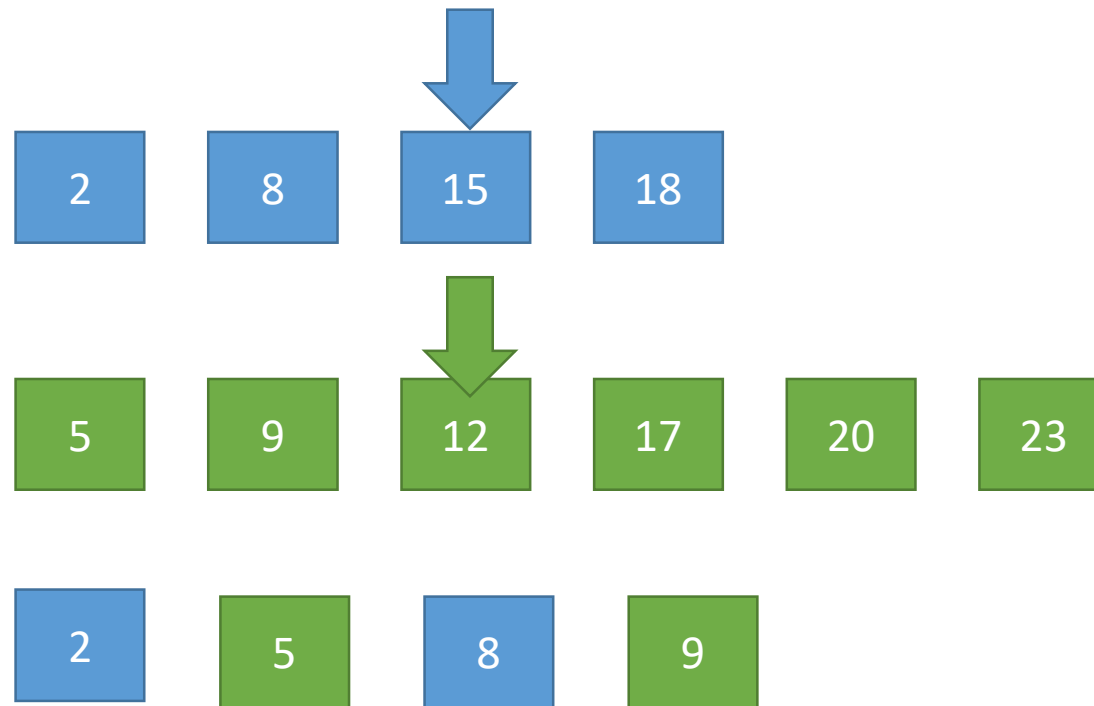
Merging Two Decks of Cards



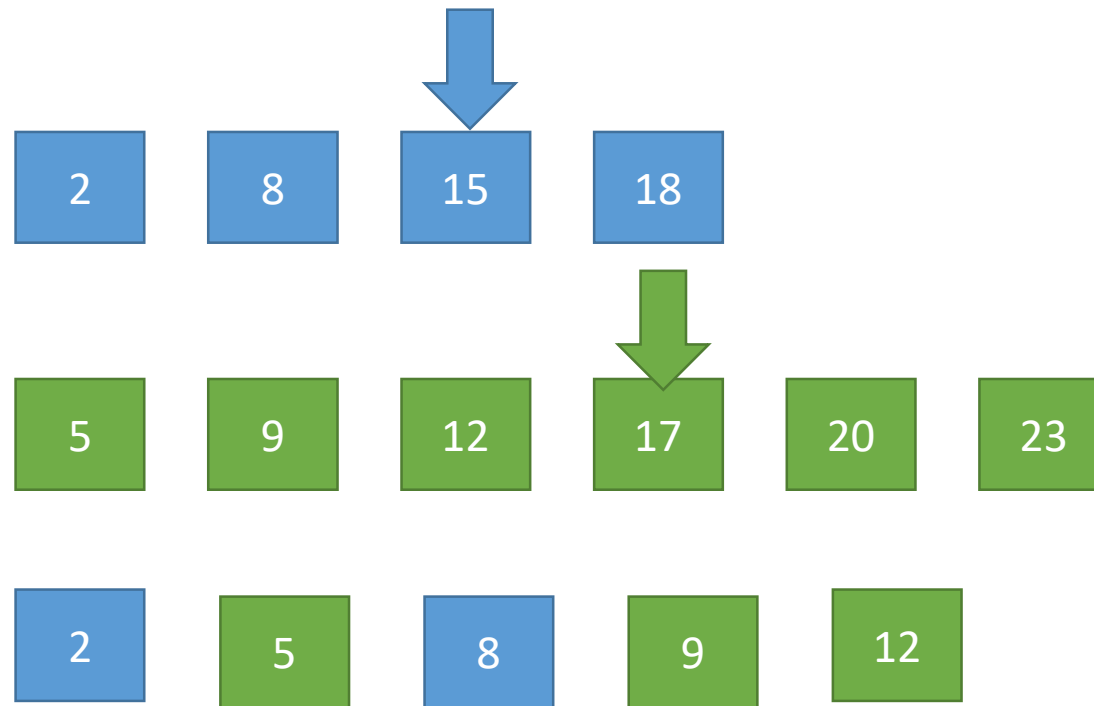
Merging Two Decks of Cards



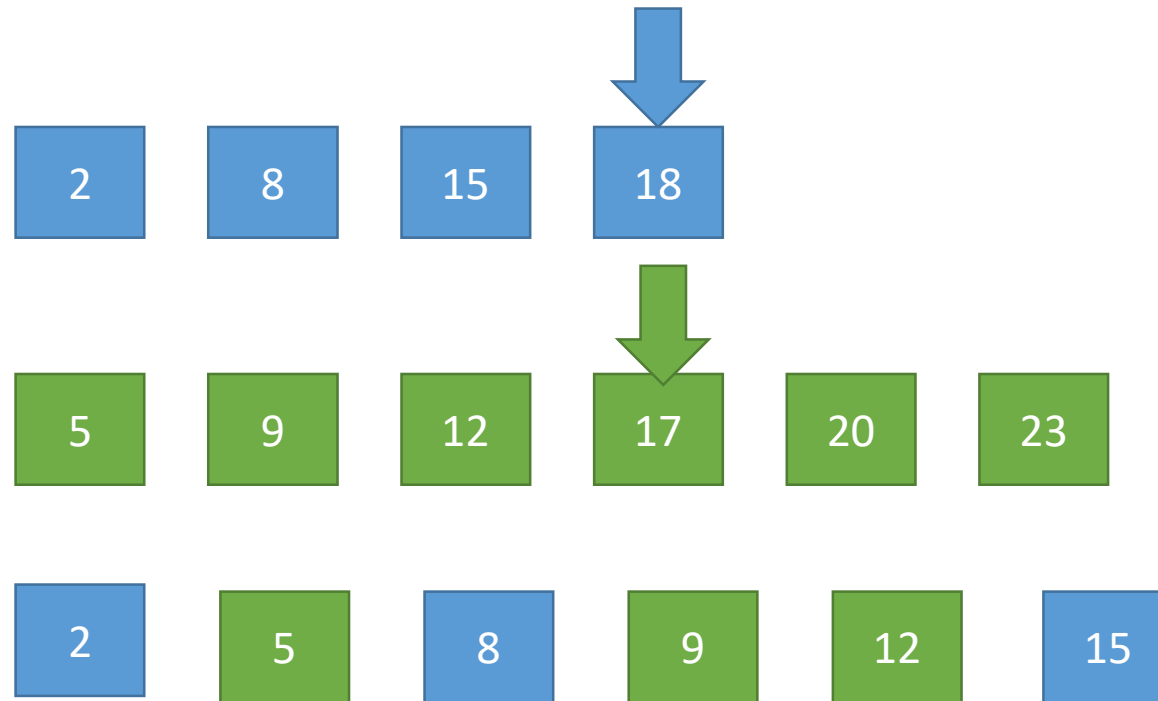
Merging Two Decks of Cards



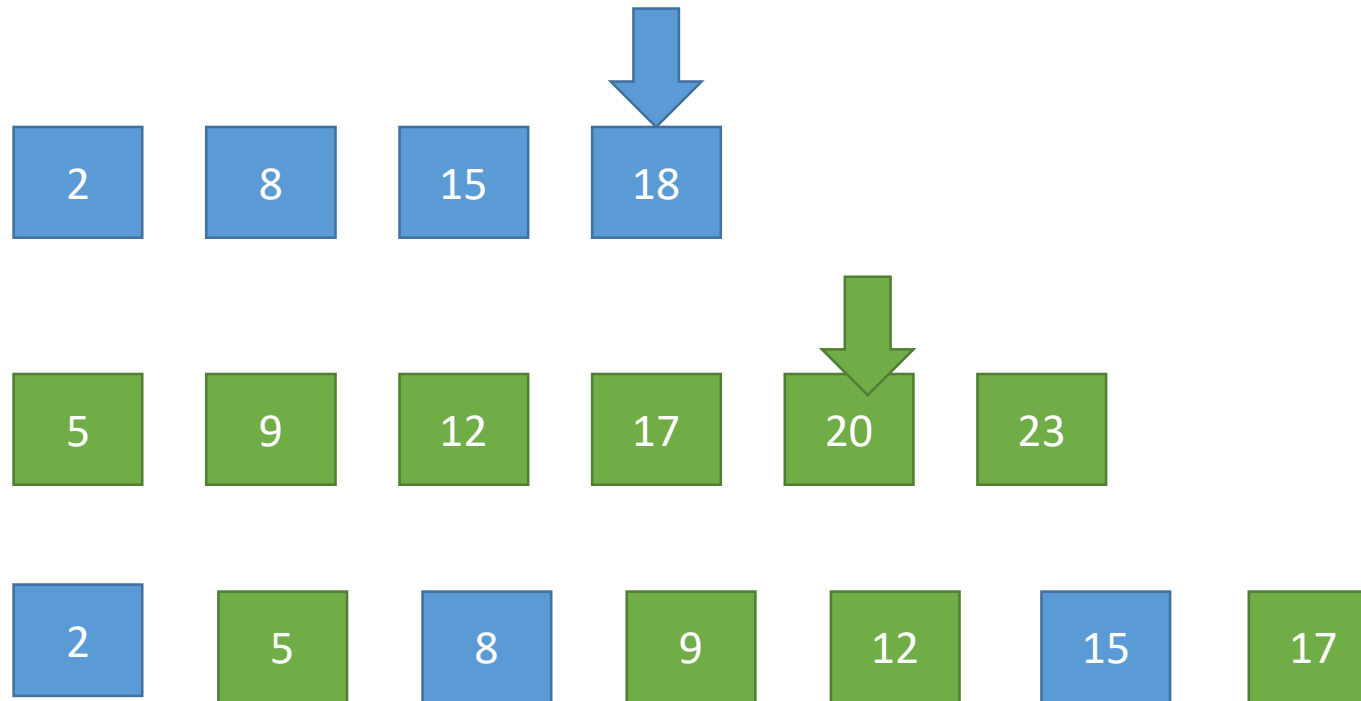
Merging Two Decks of Cards



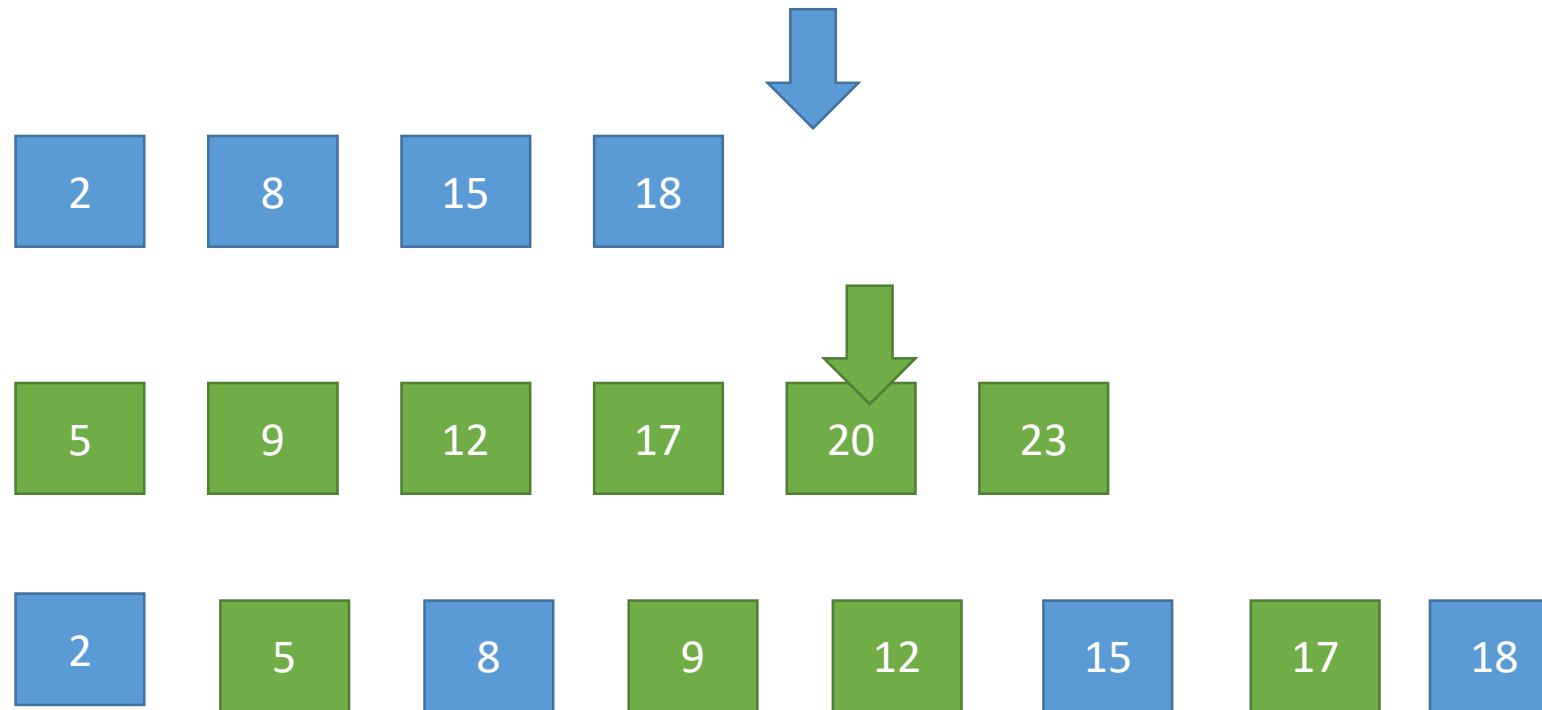
Merging Two Decks of Cards



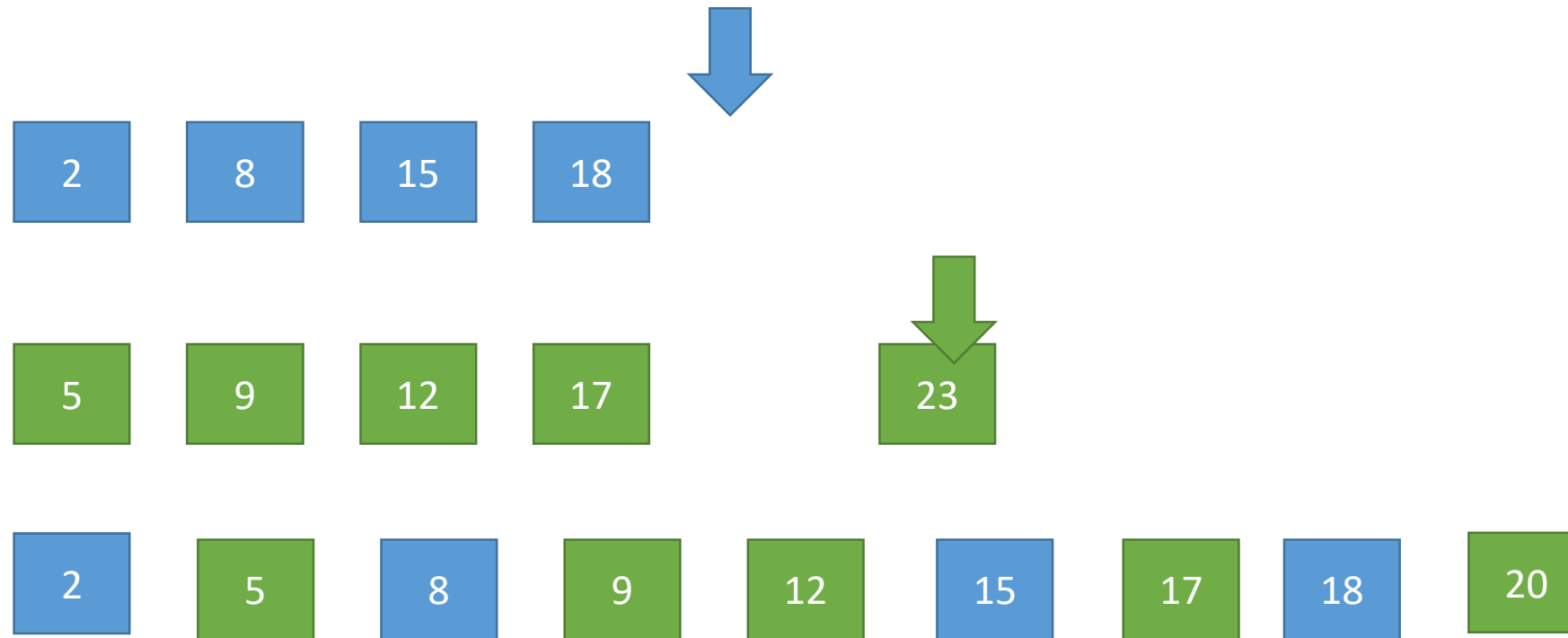
Merging Two Decks of Cards



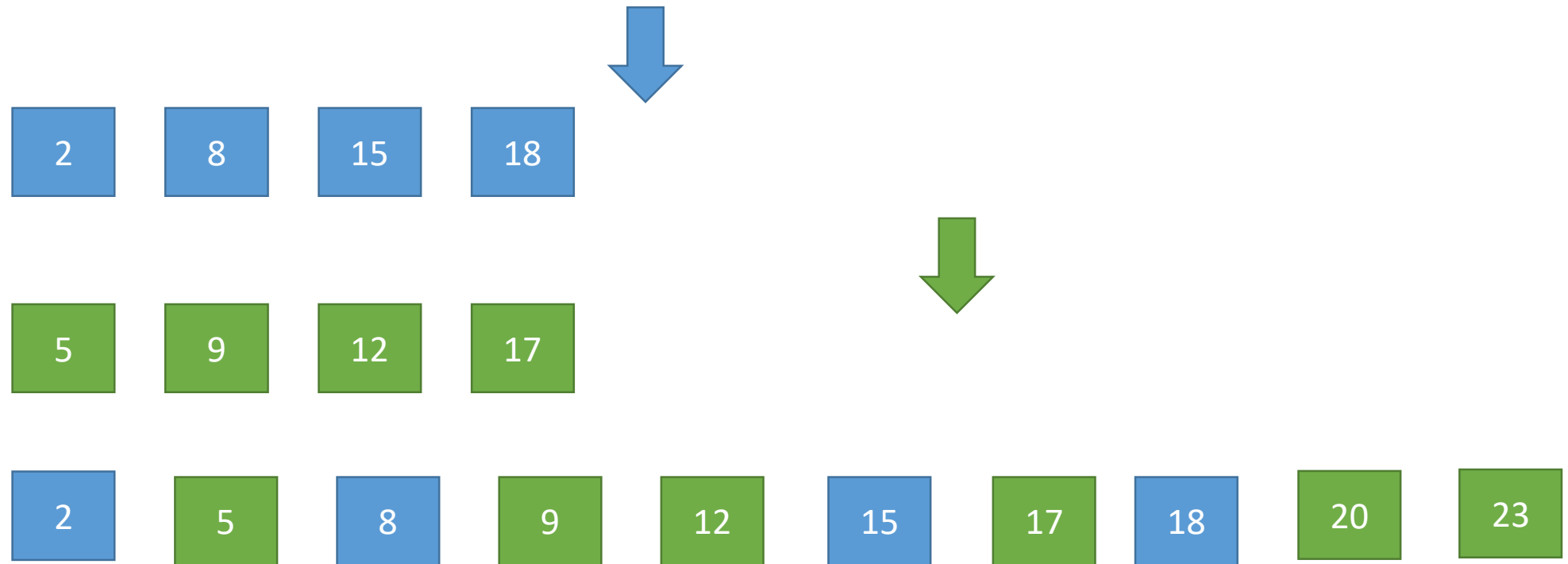
Merging Two Decks of Cards



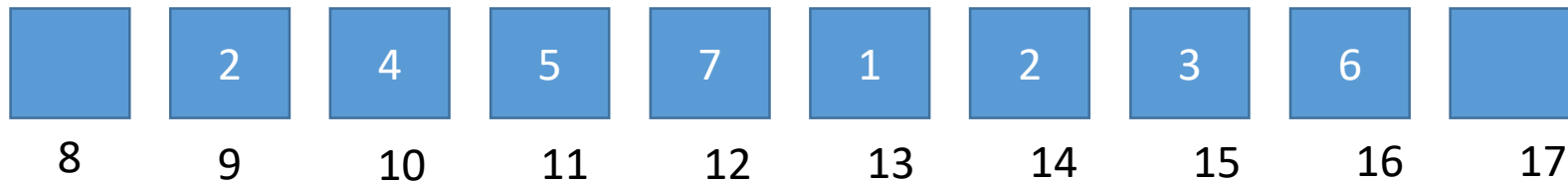
Merging Two Decks of Cards



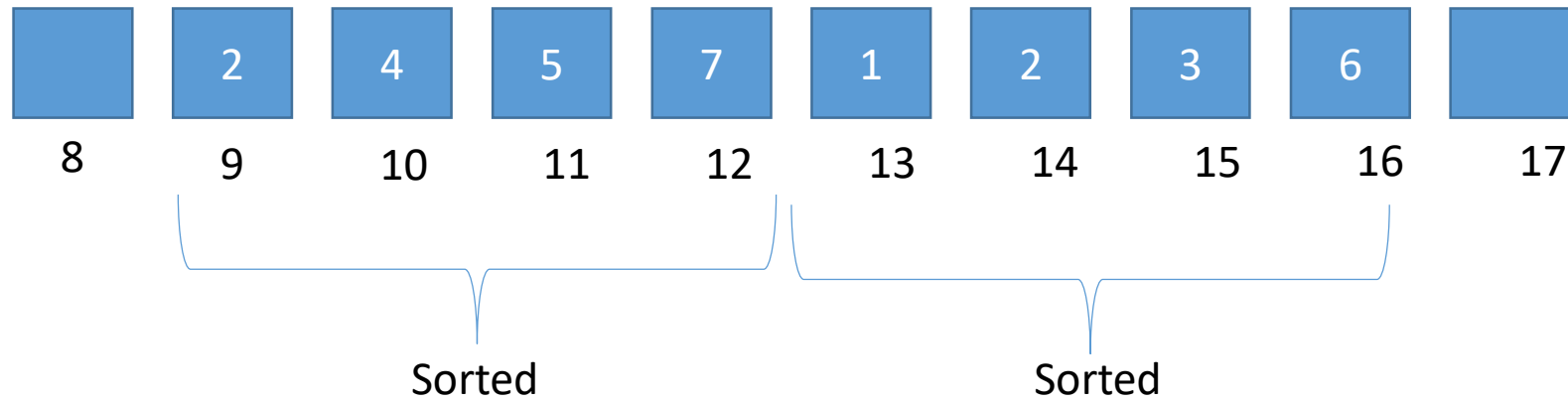
Merging Two Decks of Cards



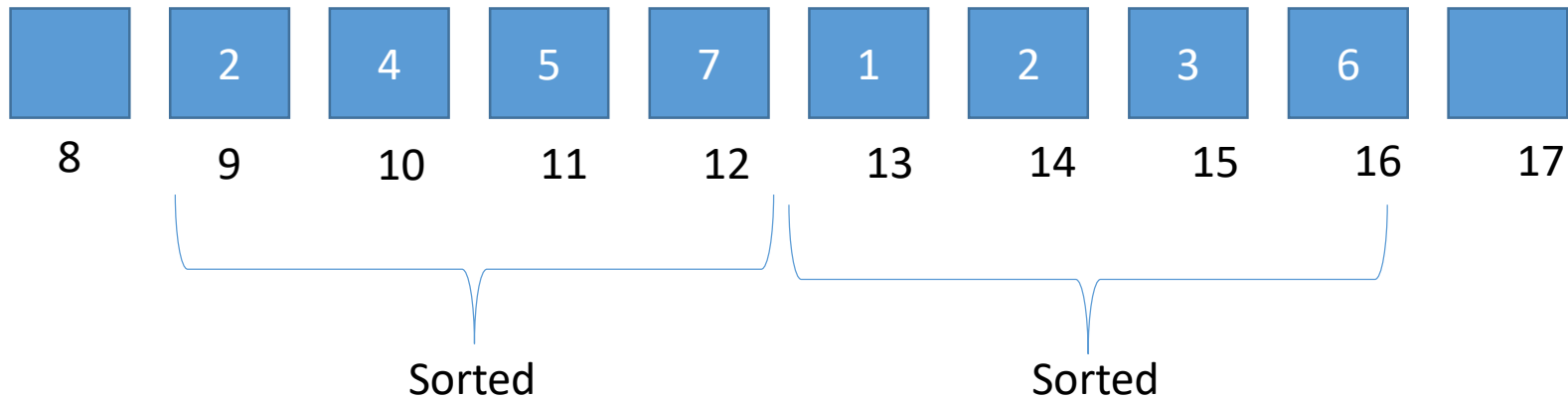
Merging in an Array



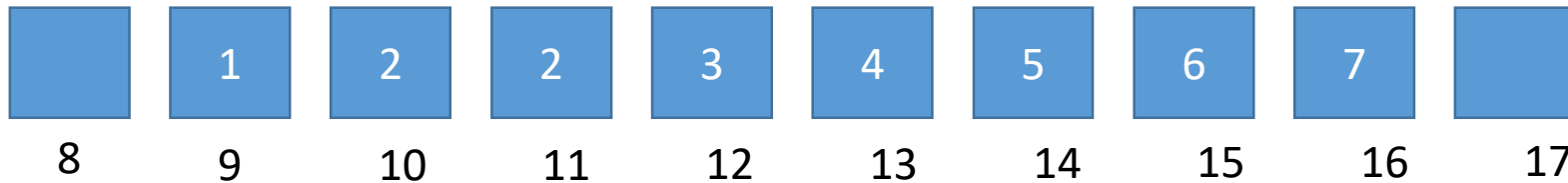
Merging in an Array



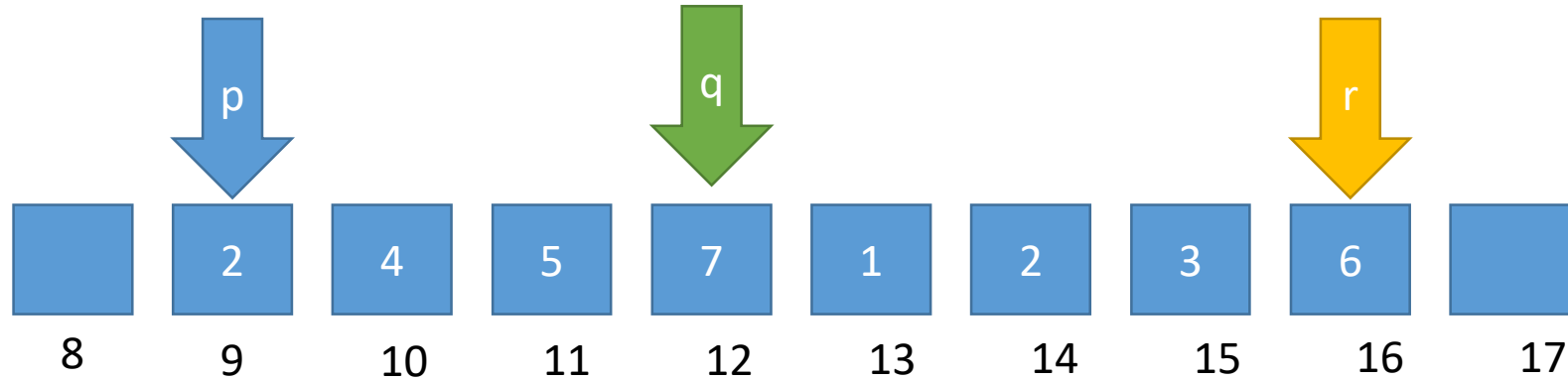
Merging in an Array



What we want:

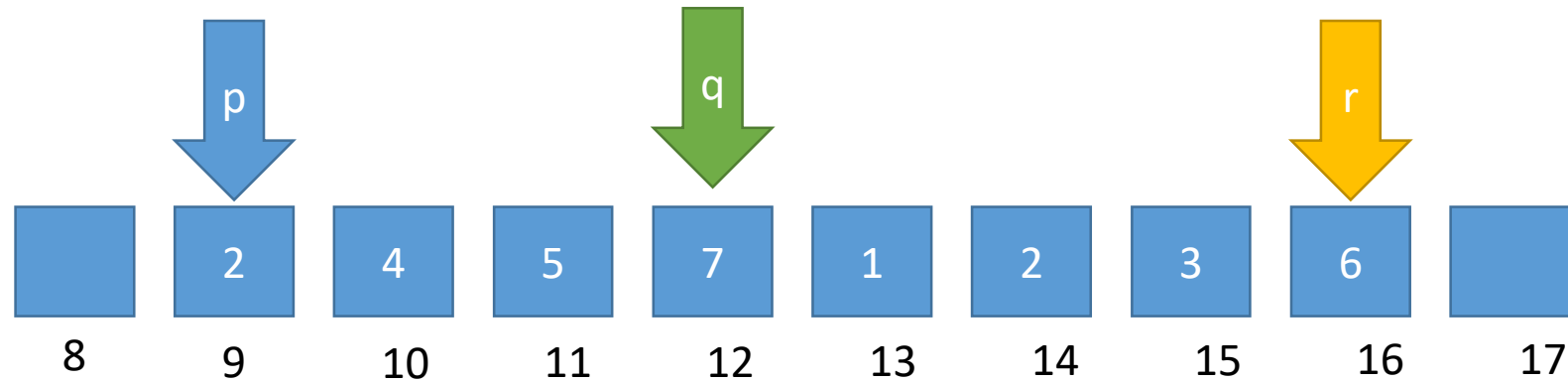


Merging in an Array



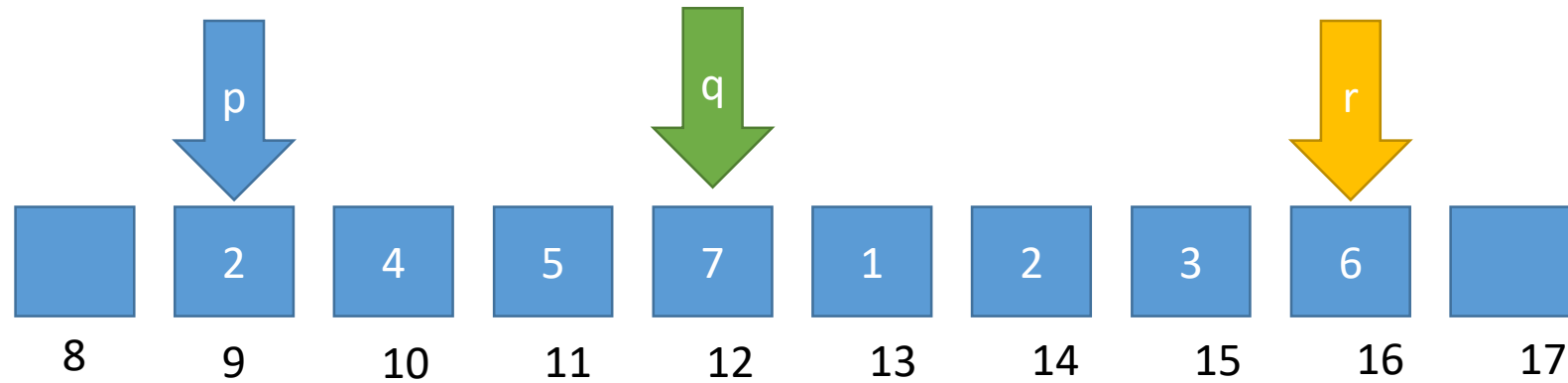
p, q and r were given to us along with the array

Merging in an Array



We want to make **two new arrays**.
Each with the sorted elements only.

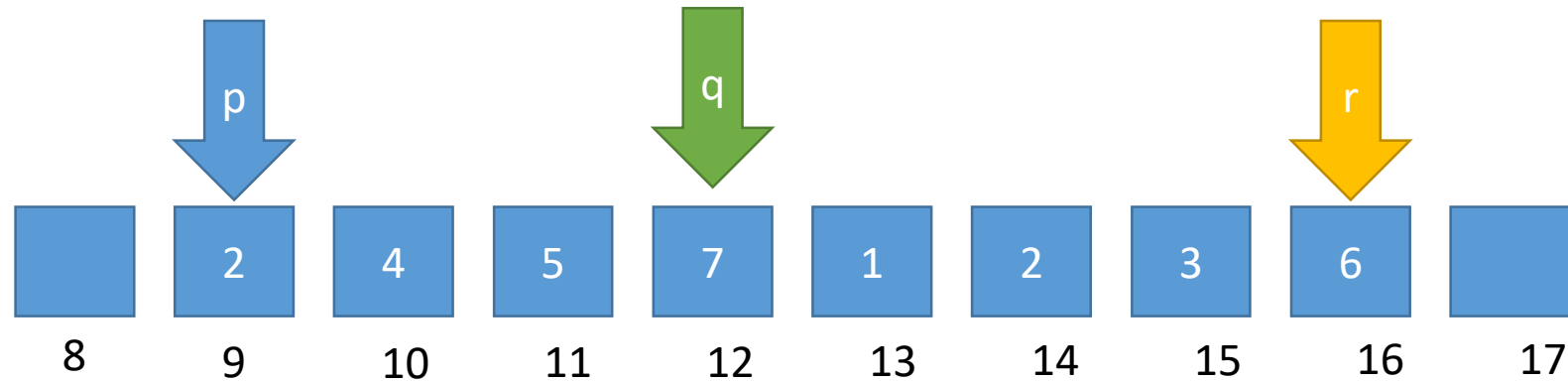
Merging in an Array



$$n_1 = q - p + 1 = 12 - 9 + 1 = 4$$

$$n_2 = r - q = 16 - 12 = 4$$

Merging in an Array

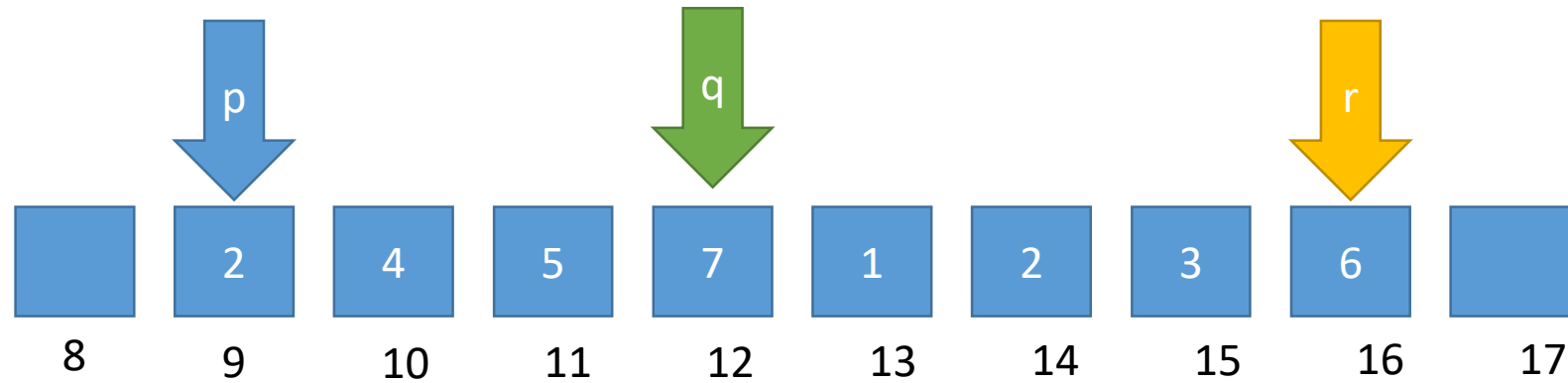


$$n_1 = q - p + 1 = 12 - 9 + 1 = 4$$

$$n_2 = r - q = 16 - 12 = 4$$



Merging in an Array



$$n_1 = q - p + 1 = 12 - 9 + 1 = 4$$

$$n_2 = r - q = 16 - 12 = 4$$

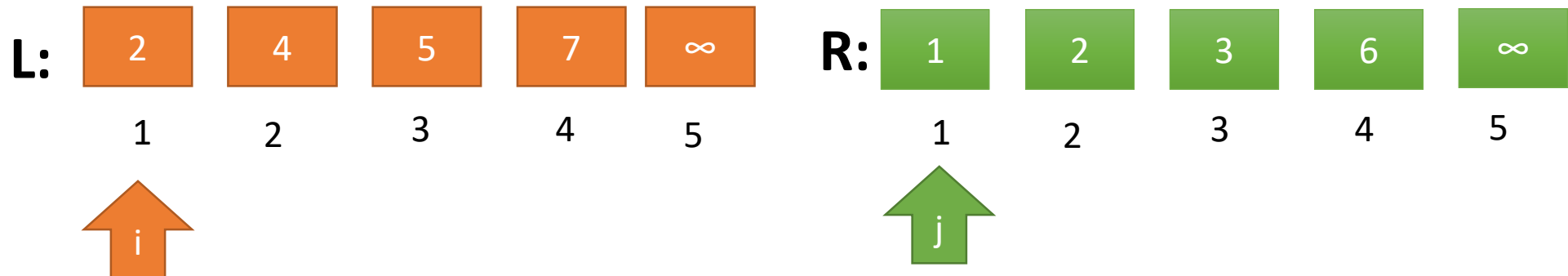


Merging in an Array



$$n_1 = q - p + 1 = 12 - 9 + 1 = 4$$

$$n_2 = r - q = 16 - 12 = 4$$

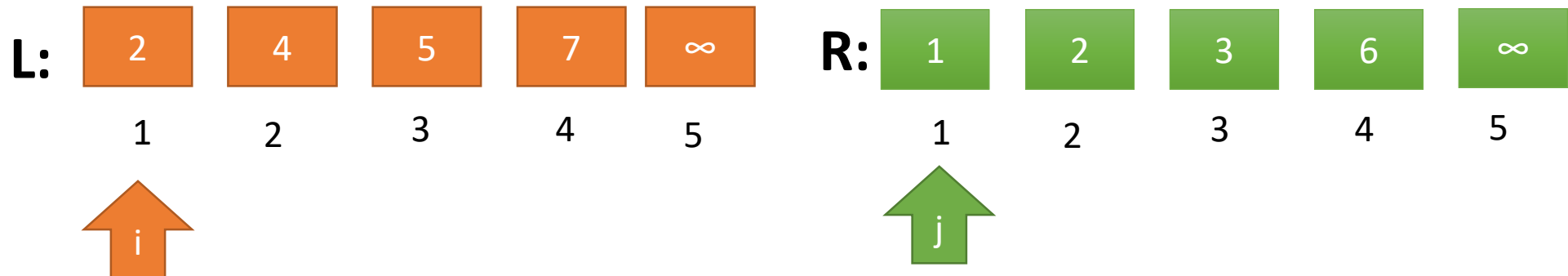


Merging in an Array

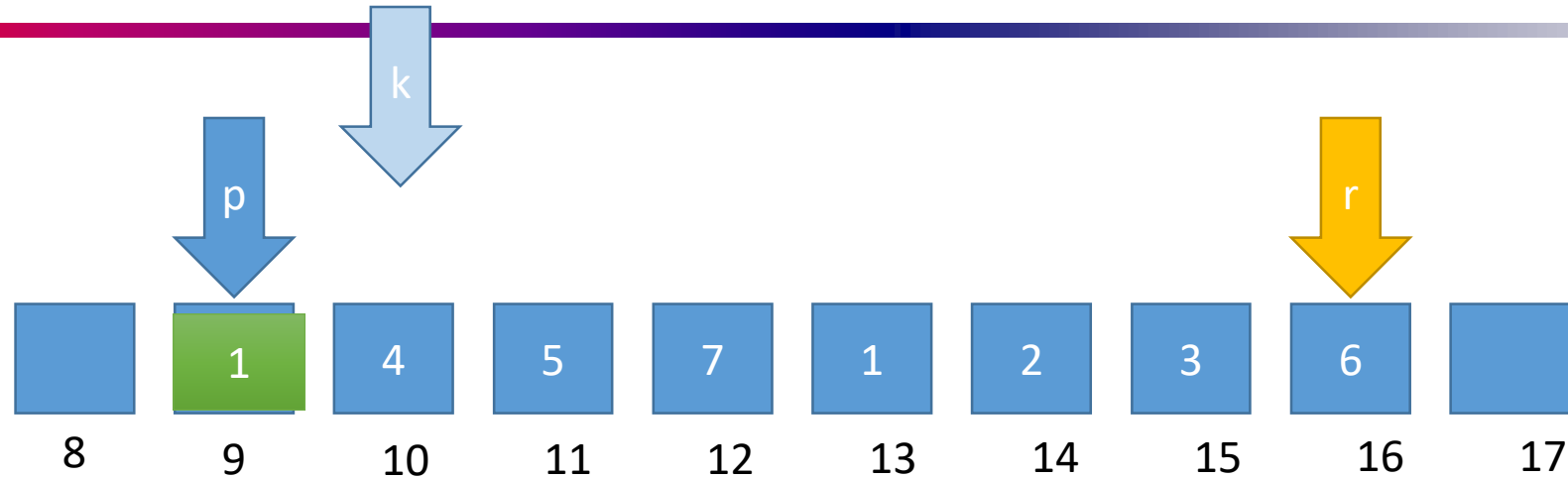


$$n_1 = q - p + 1 = 12 - 9 + 1 = 4$$

$$n_2 = r - q = 16 - 12 = 4$$



Merging in an Array

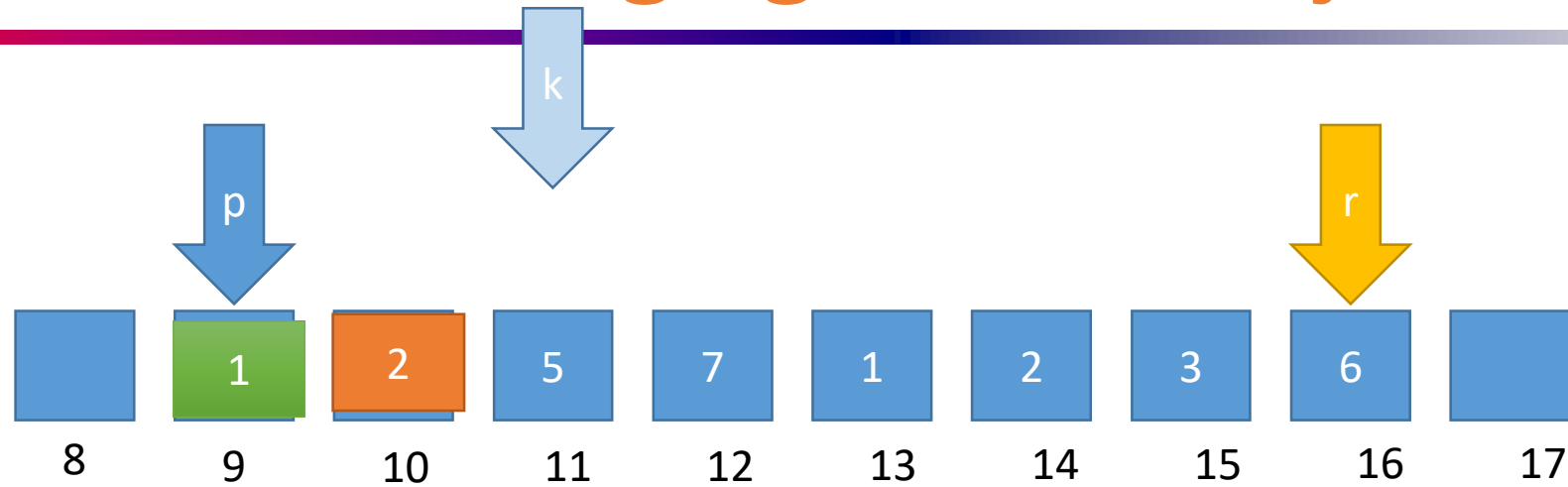


$$n_1 = q - p + 1 = 12 - 9 + 1 = 4$$

$$n_2 = r - q = 16 - 12 = 4$$

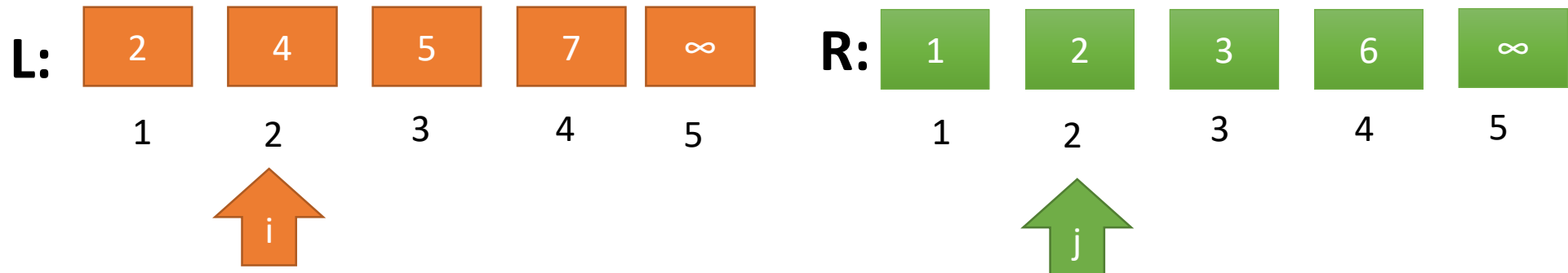


Merging in an Array

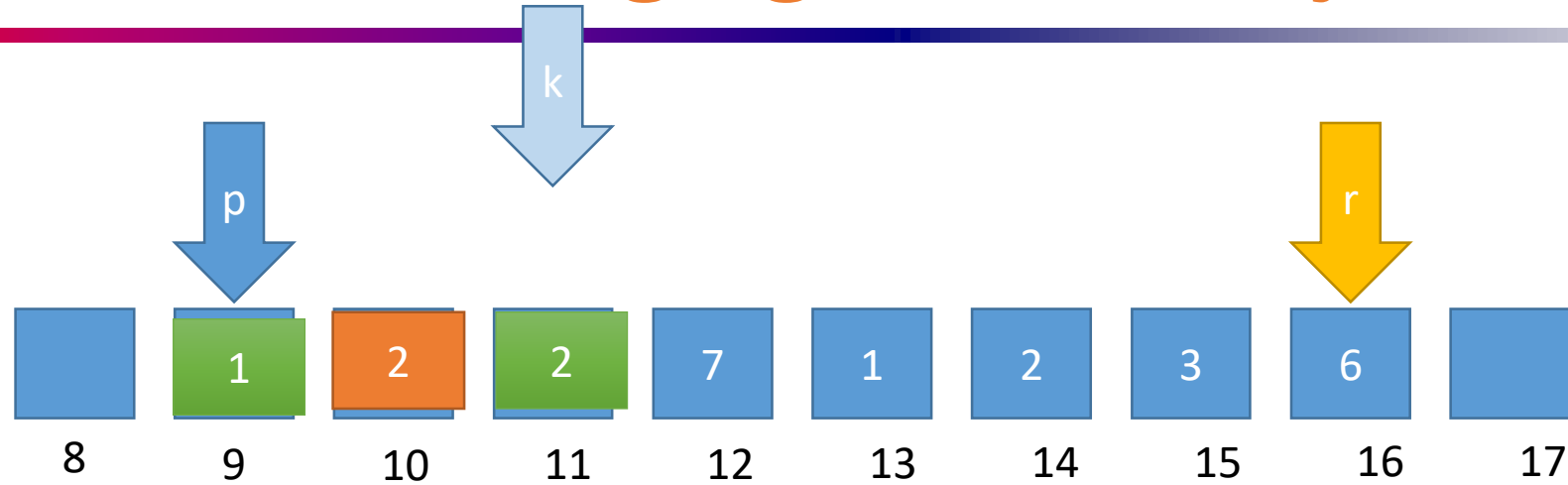


$$n_1 = q - p + 1 = 12 - 9 + 1 = 4$$

$$n_2 = r - q = 16 - 12 = 4$$

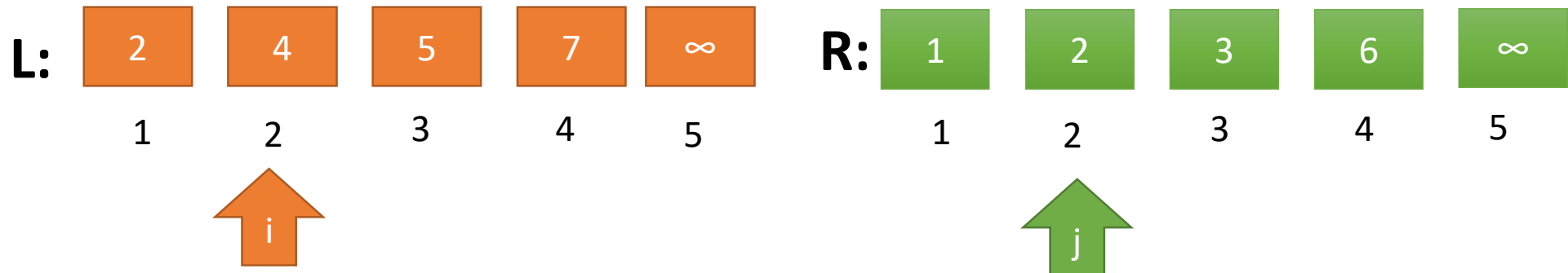


Merging in an Array

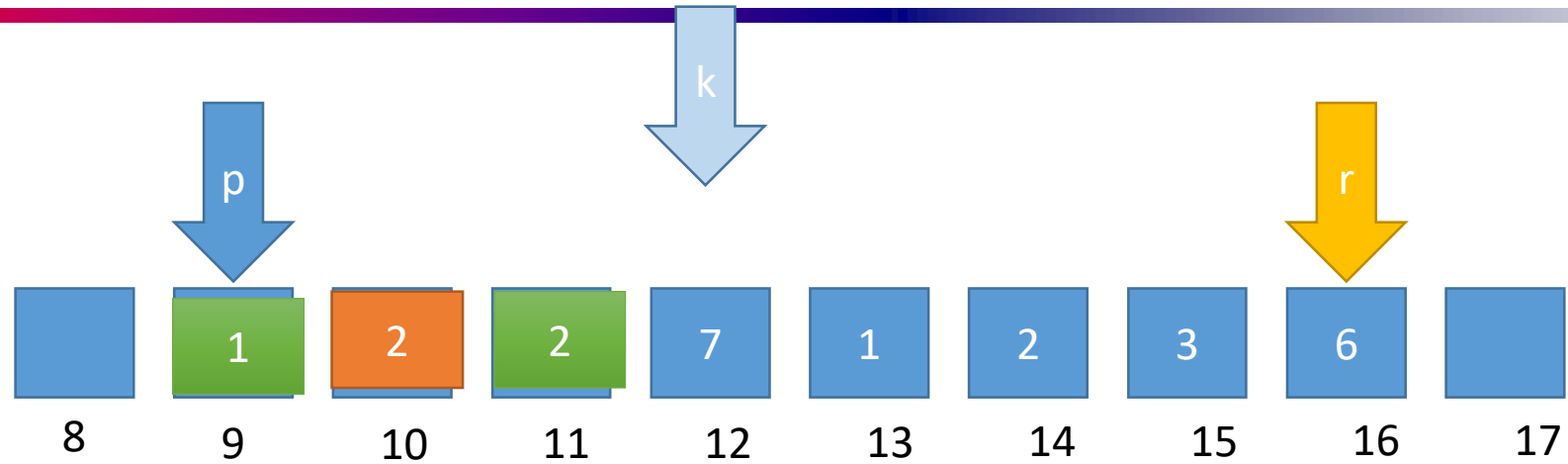


$$n_1 = q - p + 1 = 12 - 9 + 1 = 4$$

$$n_2 = r - q = 16 - 12 = 4$$

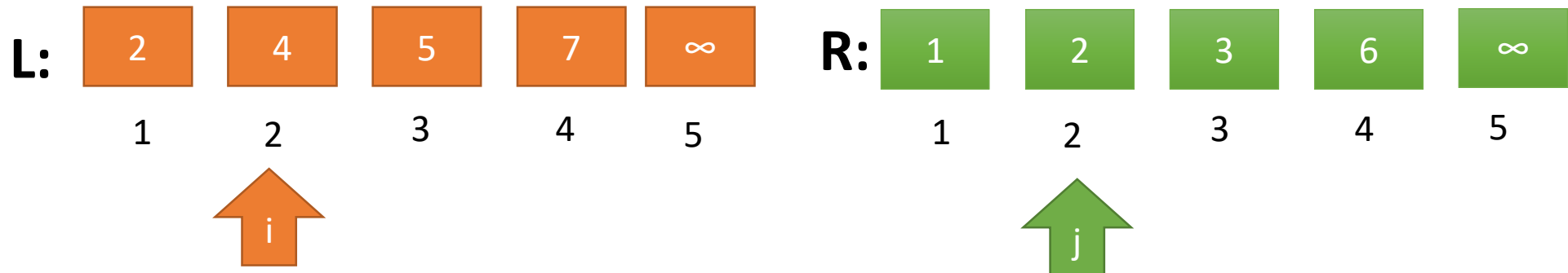


Merging in an Array

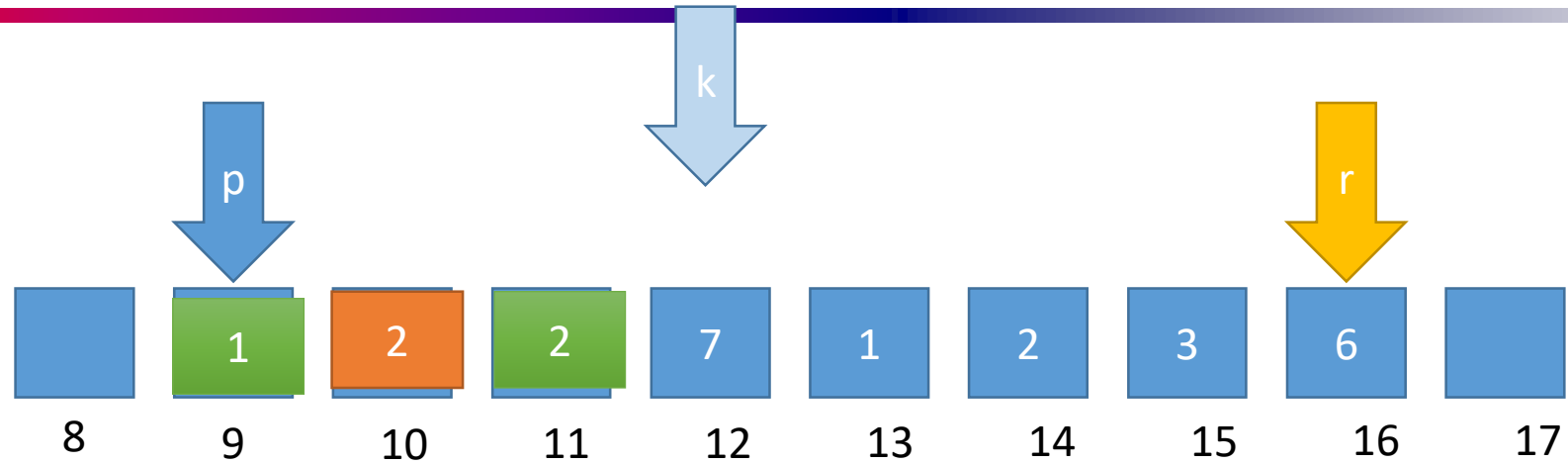


$$n_1 = q - p + 1 = 12 - 9 + 1 = 4$$

$$n_2 = r - q = 16 - 12 = 4$$



Merging in an Array

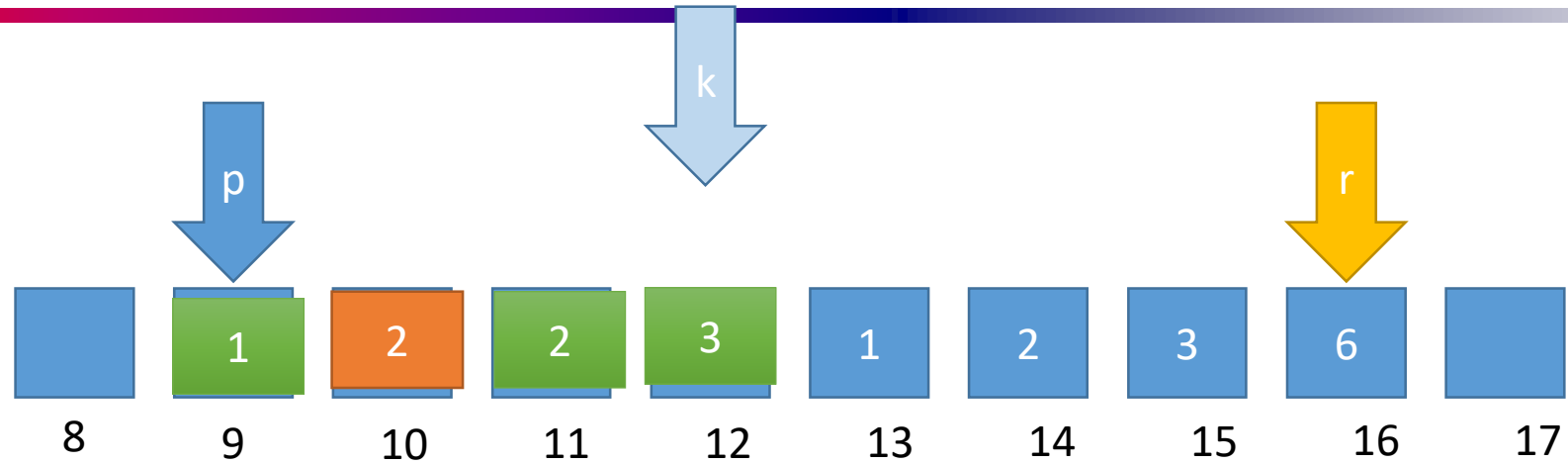


$$n_1 = q - p + 1 = 12 - 9 + 1 = 4$$

$$n_2 = r - q = 16 - 12 = 4$$



Merging in an Array

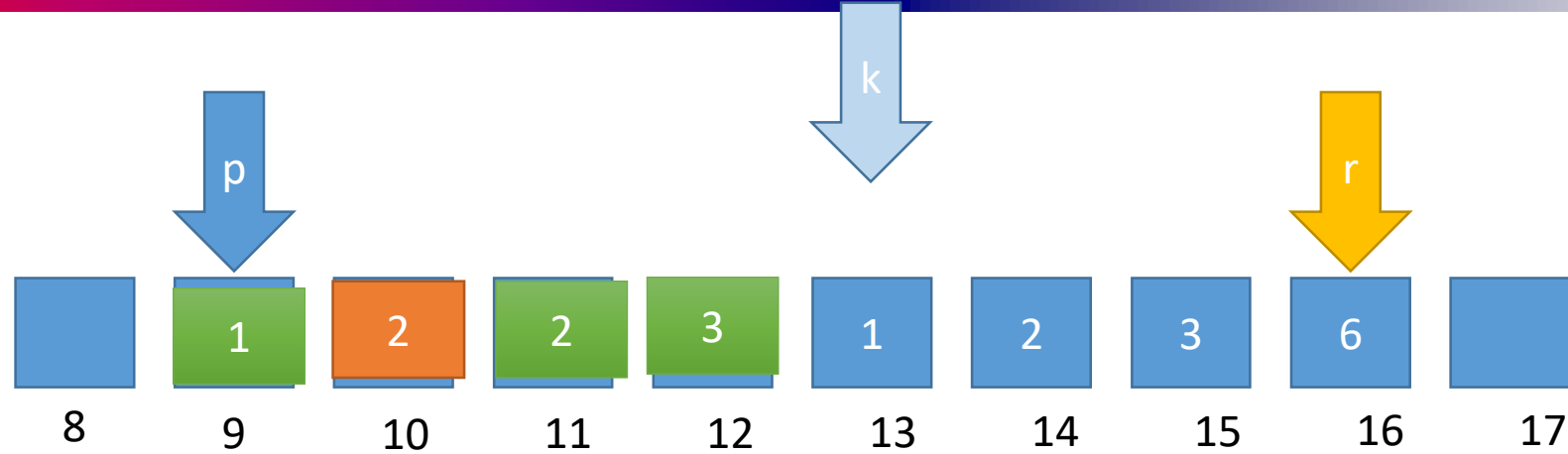


$$n_1 = q - p + 1 = 12 - 9 + 1 = 4$$

$$n_2 = r - q = 16 - 12 = 4$$



Merging in an Array

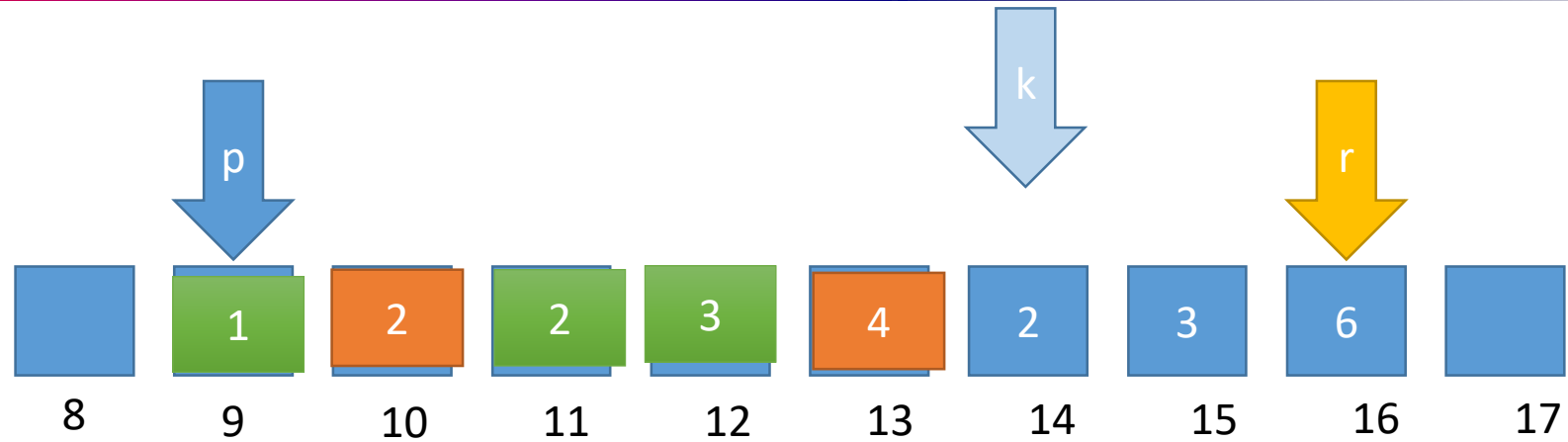


$$n_1 = q - p + 1 = 12 - 9 + 1 = 4$$

$$n_2 = r - q = 16 - 12 = 4$$



Merging in an Array

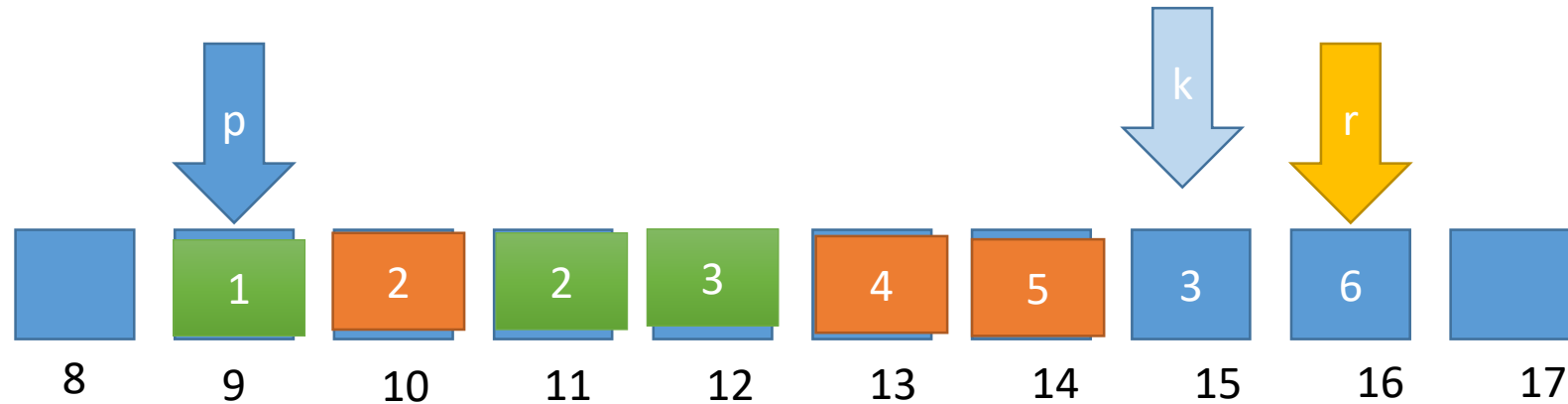


$$n_1 = q - p + 1 = 12 - 9 + 1 = 4$$

$$n_2 = r - q = 16 - 12 = 4$$



Merging in an Array

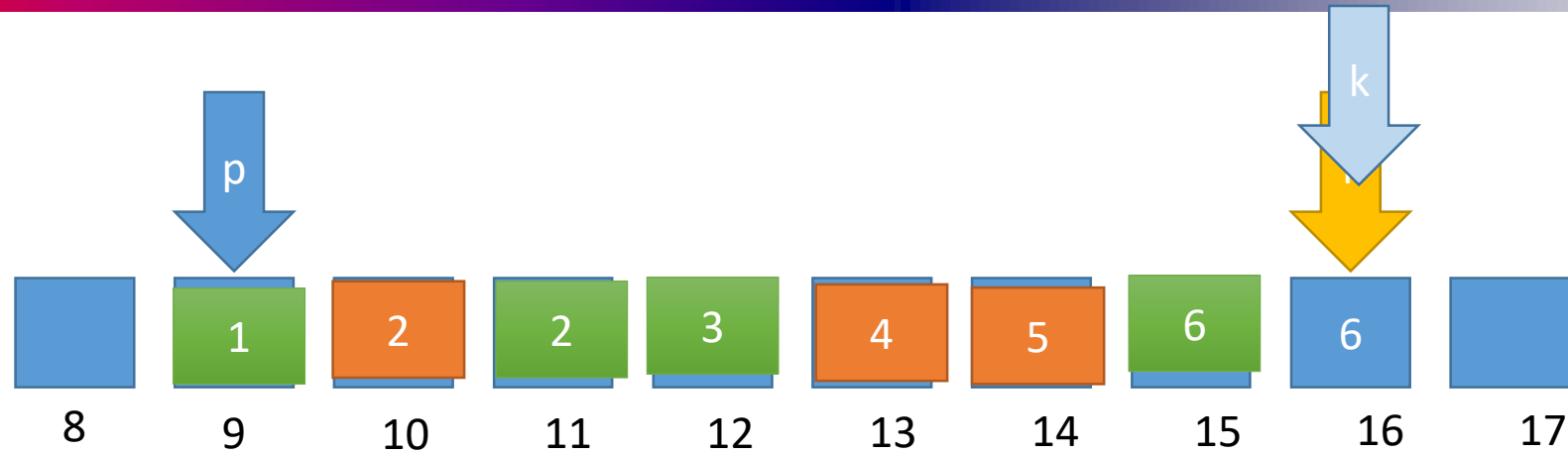


$$n_1 = q - p + 1 = 12 - 9 + 1 = 4$$

$$n_2 = r - q = 16 - 12 = 4$$



Merging in an Array

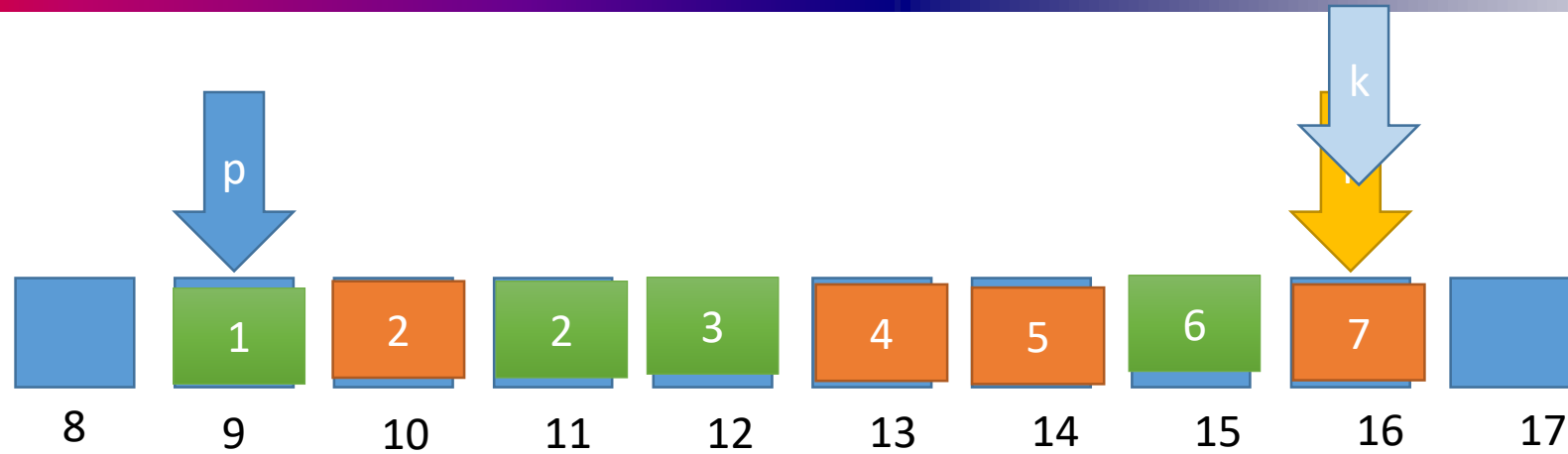


$$n_1 = q - p + 1 = 12 - 9 + 1 = 4$$

$$n_2 = r - q = 16 - 12 = 4$$



Merging in an Array

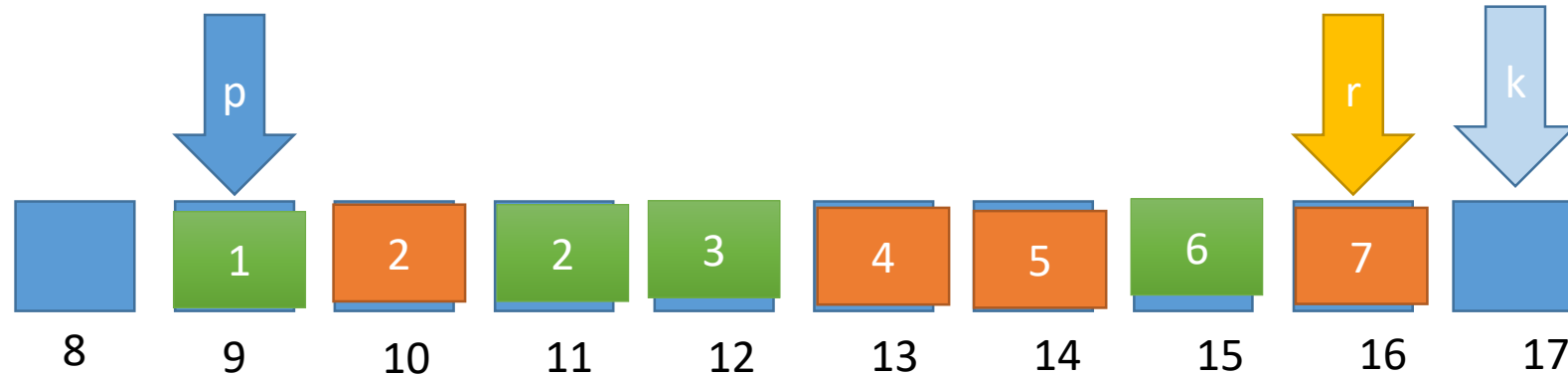


$$n_1 = q - p + 1 = 12 - 9 + 1 = 4$$

$$n_2 = r - q = 16 - 12 = 4$$

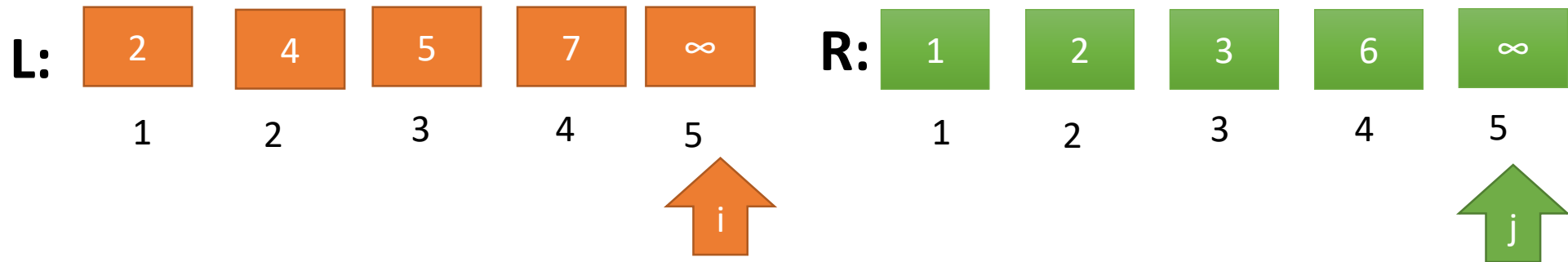


Merging in an Array



$$n_1 = q - p + 1 = 12 - 9 + 1 = 4$$

$$n_2 = r - q = 16 - 12 = 4$$



Merge Pseudocode

- Note that merging takes $\Theta(n)$ time

```
MERGE( $A, p, q, r$ )
1   $n_1 = q - p + 1$ 
2   $n_2 = r - q$ 
3  let  $L[1..n_1 + 1]$  and  $R[1..n_2 + 1]$  be new arrays
4  for  $i = 1$  to  $n_1$ 
5       $L[i] = A[p + i - 1]$ 
6  for  $j = 1$  to  $n_2$ 
7       $R[j] = A[q + j]$ 
8   $L[n_1 + 1] = \infty$ 
9   $R[n_2 + 1] = \infty$ 
10  $i = 1$ 
11  $j = 1$ 
12 for  $k = p$  to  $r$ 
13     if  $L[i] \leq R[j]$ 
14          $A[k] = L[i]$ 
15          $i = i + 1$ 
16     else  $A[k] = R[j]$ 
17          $j = j + 1$ 
```

Merge Pseudocode

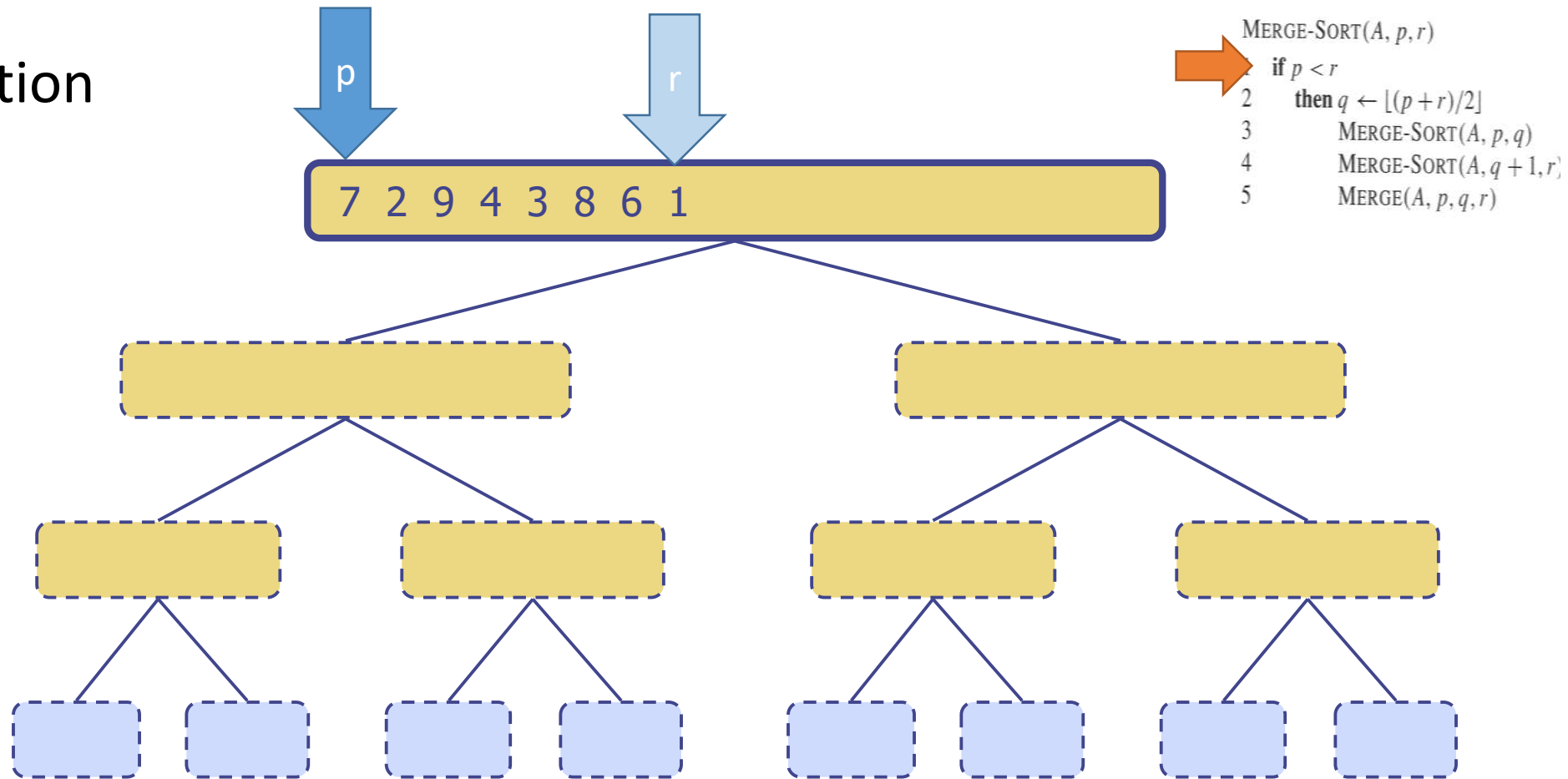
```
MERGE( $A, p, q, r$ )  
1   $n_1 = q - p + 1$   
2   $n_2 = r - q$   
3  let  $L[1..n_1 + 1]$  and  $R[1..n_2 + 1]$  be new arrays  
4  for  $i = 1$  to  $n_1$   
5       $L[i] = A[p + i - 1]$   
6  for  $j = 1$  to  $n_2$   
7       $R[j] = A[q + j]$   
8   $L[n_1 + 1] = \infty$   
9   $R[n_2 + 1] = \infty$   
10  $i = 1$   
11  $j = 1$   
12 for  $k = p$  to  $r$   
13     if  $L[i] \leq R[j]$   
14          $A[k] = L[i]$   
15          $i = i + 1$   
16     else  $A[k] = R[j]$   
17          $j = j + 1$ 
```

Merge Sort Pseudocode

```
MERGE-SORT( $A, p, r$ )  
1  if  $p < r$   
2      then  $q \leftarrow \lfloor (p + r)/2 \rfloor$   
3          MERGE-SORT( $A, p, q$ )  
4          MERGE-SORT( $A, q + 1, r$ )  
5          MERGE( $A, p, q, r$ )
```

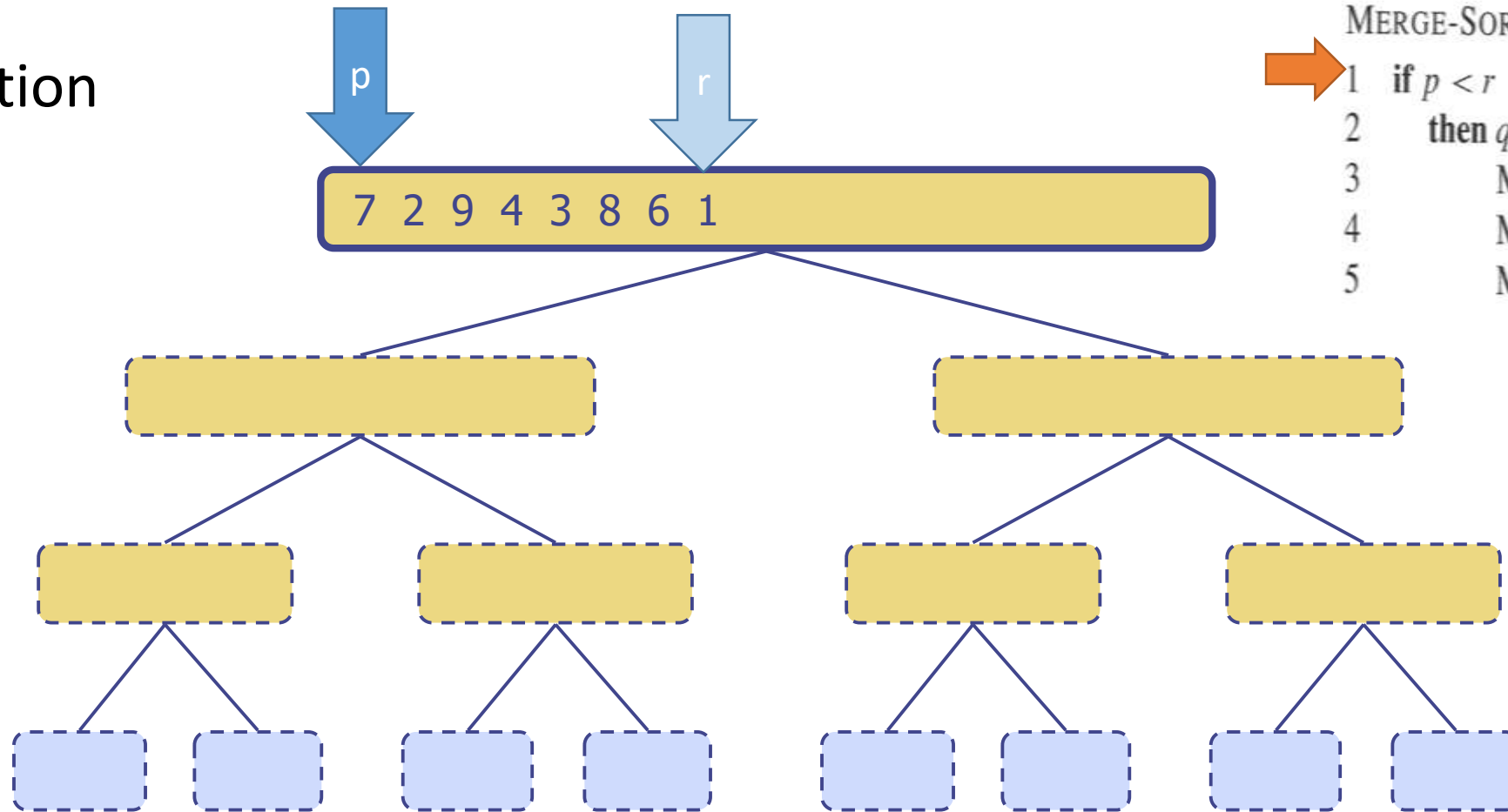
Merge Sort Simulation

- Partition



Merge Sort Simulation

- Partition

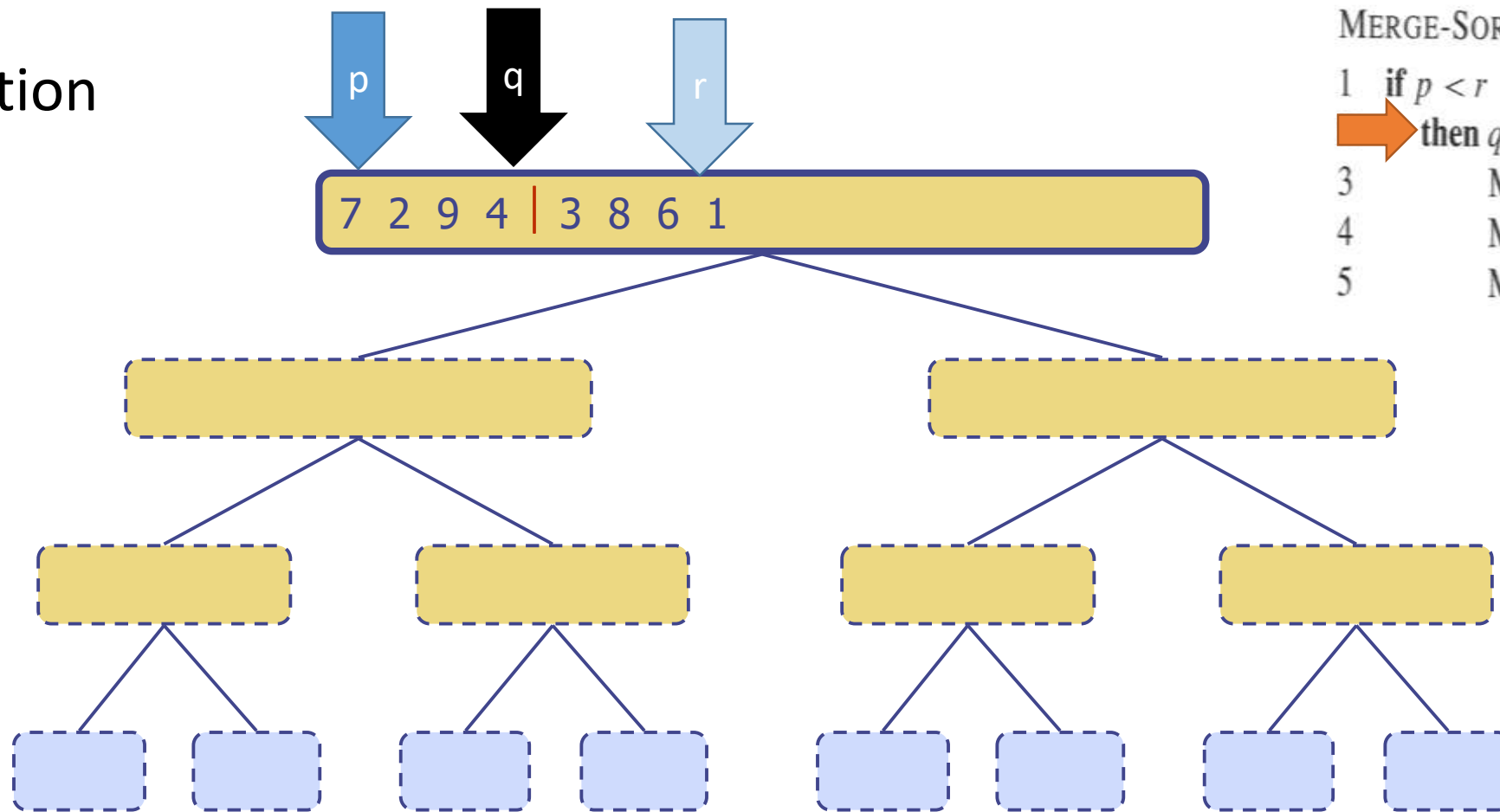


MERGE-SORT(A, p, r)

1 **if** $p < r$
2 **then** $q \leftarrow \lfloor (p + r) / 2 \rfloor$
3 MERGE-SORT(A, p, q)
4 MERGE-SORT($A, q + 1, r$)
5 MERGE(A, p, q, r)

Merge Sort Simulation

- Partition



```
MERGE-SORT( $A, p, r$ )
1  if  $p < r$ 
   then  $q \leftarrow \lfloor (p+r)/2 \rfloor$ 
3      MERGE-SORT( $A, p, q$ )
4      MERGE-SORT( $A, q+1, r$ )
5      MERGE( $A, p, q, r$ )
```

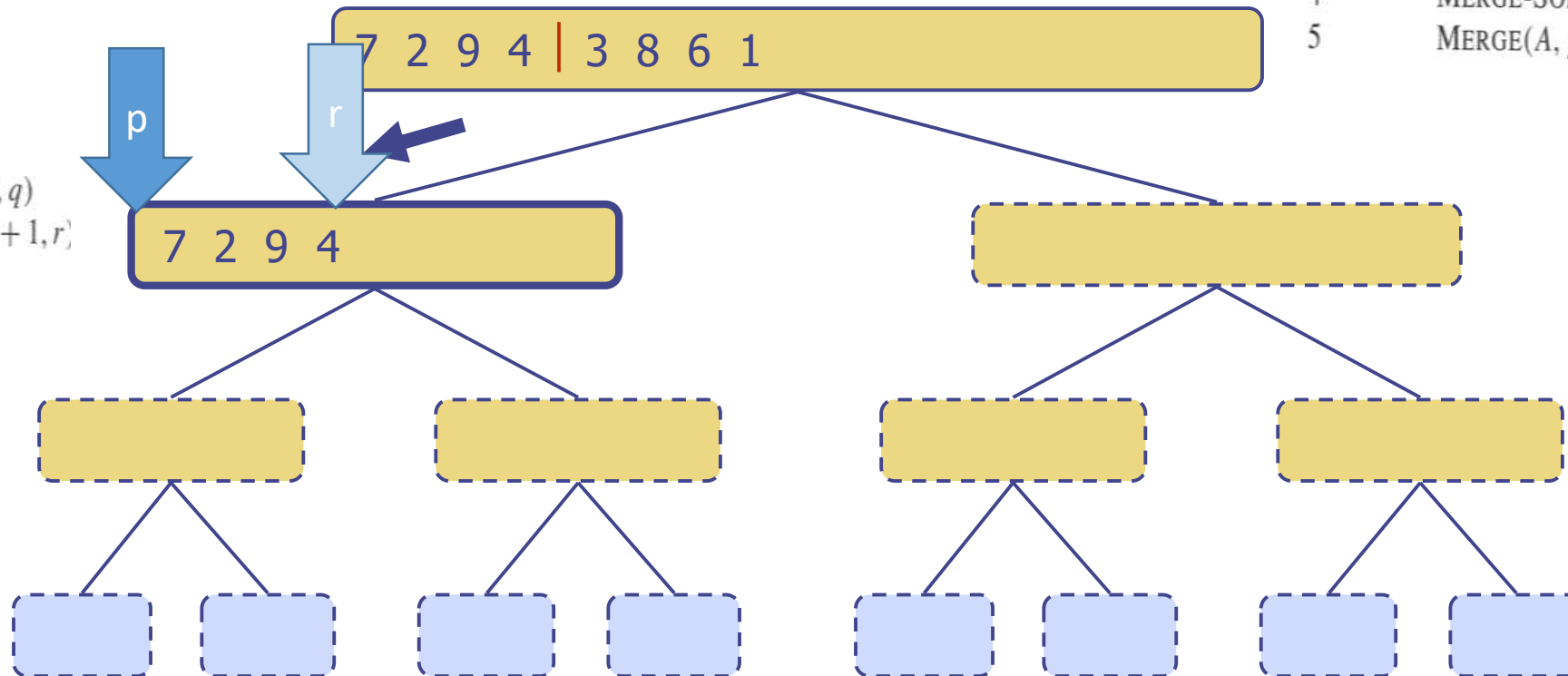
Merge Sort Simulation

- Recursive Call Partition

```
MERGE-SORT(A, p, r)
1 if p < r
2   then q ← ⌊(p+r)/2⌋
3   MERGE-SORT(A, p, q)
4   MERGE-SORT(A, q+1, r)
5   MERGE(A, p, q, r)
```

MERGE-SORT(A, p, r)

```
1 if p < r
2   then q ← ⌊(p+r)/2⌋
3   → MERGE-SORT(A, p, q)
4     MERGE-SORT(A, q+1, r)
5     MERGE(A, p, q, r)
```



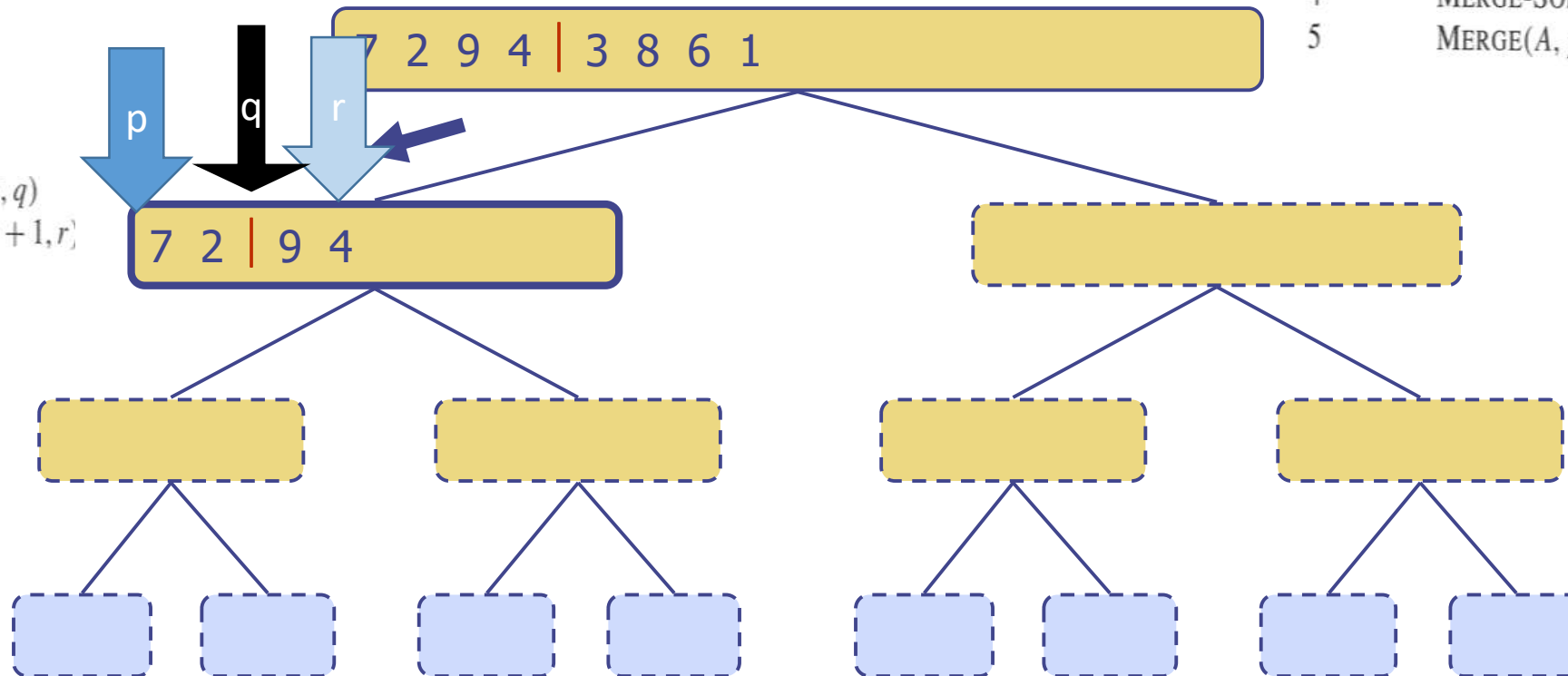
Merge Sort Simulation

- Recursive Call Partition

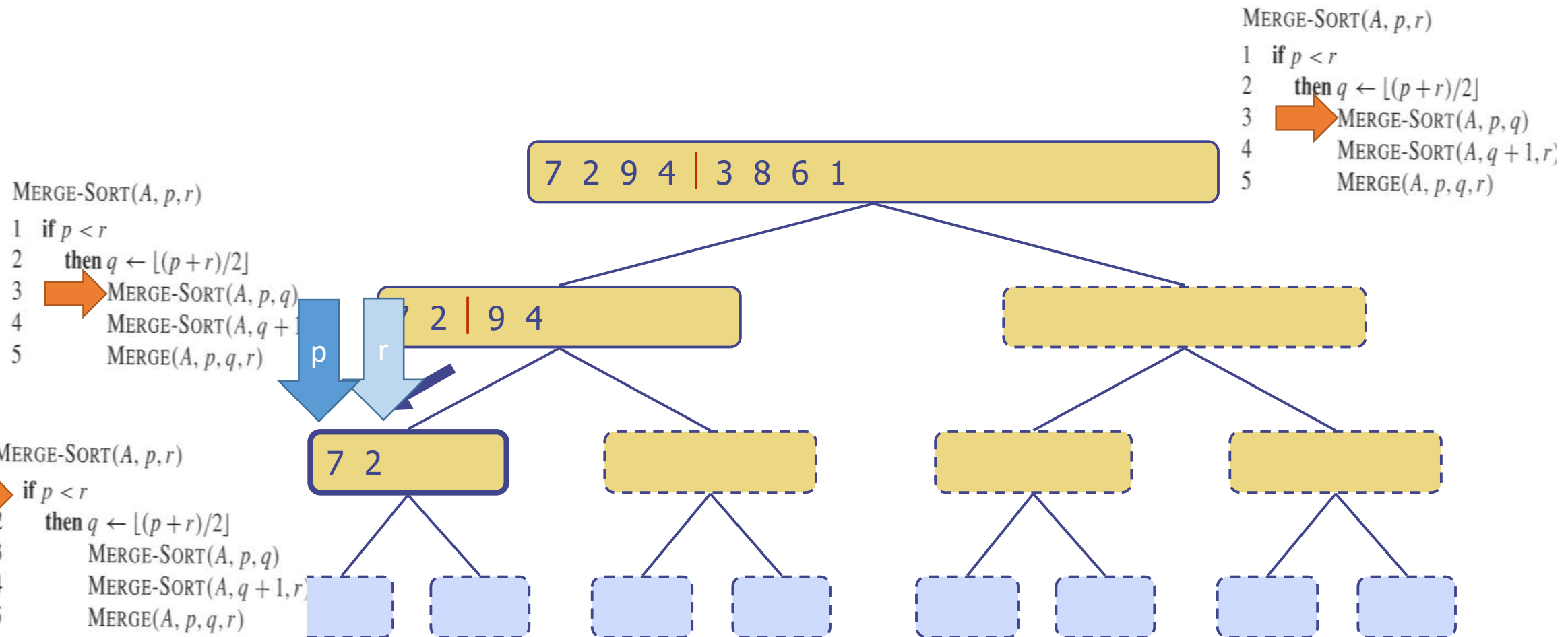
```
MERGE-SORT(A, p, r)
1  if p < r
2  then q ← ⌊(p+r)/2⌋
3  MERGE-SORT(A, p, q)
4  MERGE-SORT(A, q+1, r)
5  MERGE(A, p, q, r)
```

MERGE-SORT(A, p, r)

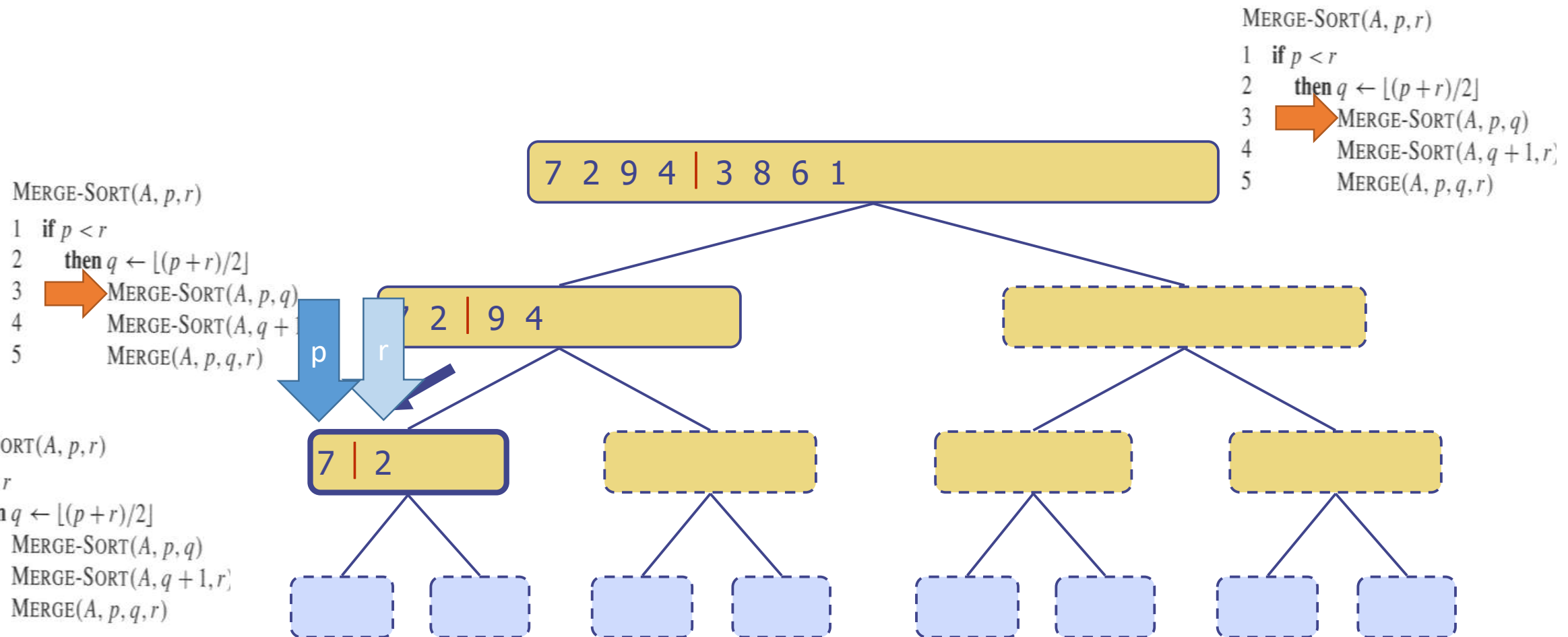
```
1  if p < r
2  then q ← ⌊(p+r)/2⌋
3  → MERGE-SORT(A, p, q)
4    MERGE-SORT(A, q+1, r)
5    MERGE(A, p, q, r)
```



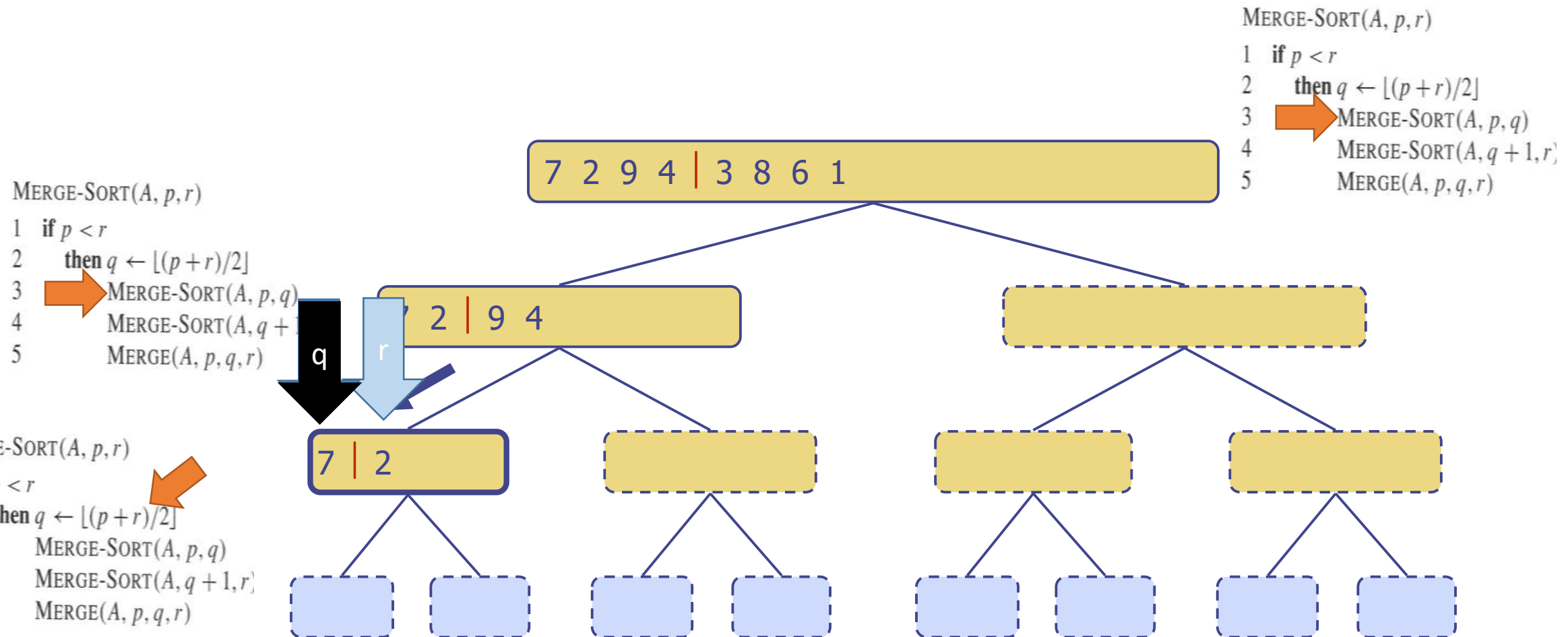
Merge Sort Simulation



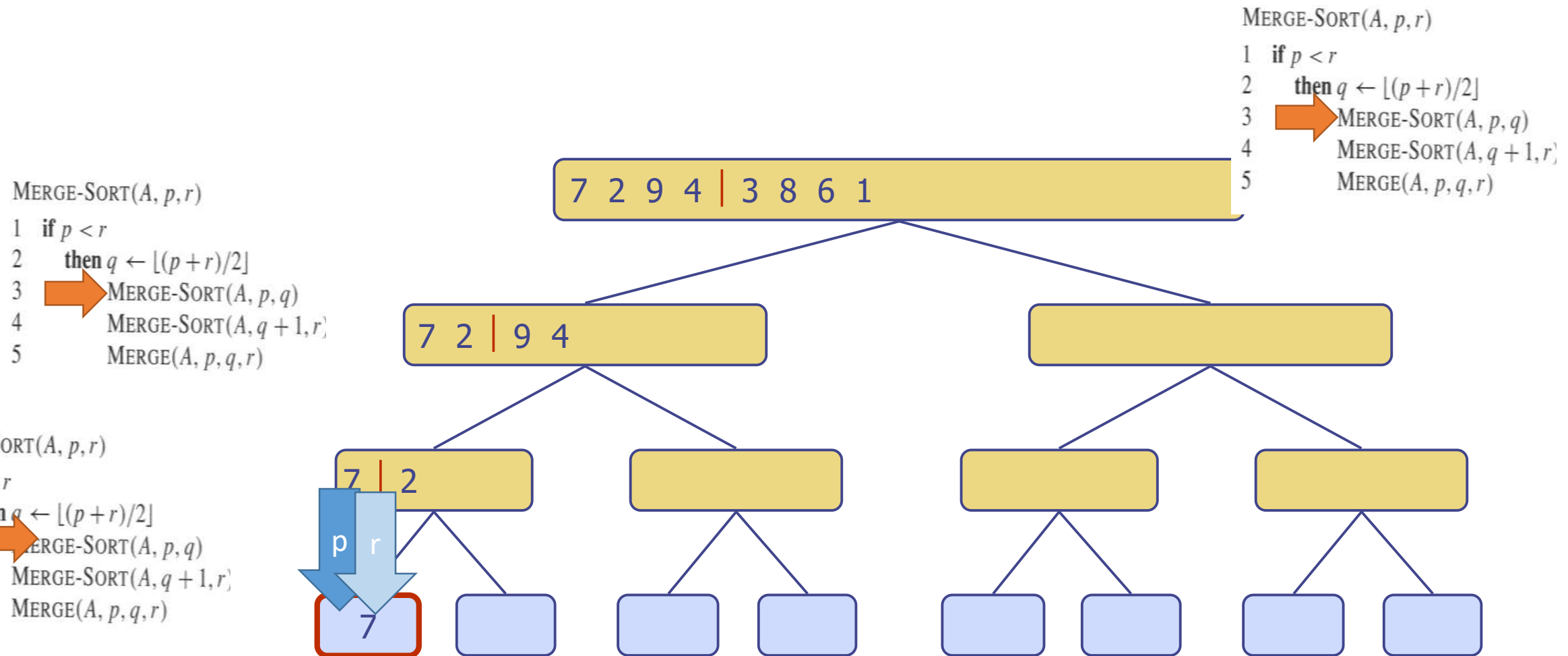
Merge Sort Simulation



Merge Sort Simulation



Merge Sort Simulation



Merge Sort Simulation

MERGE-SORT(A, p, r)

```

1 if  $p < r$ 
2   then  $q \leftarrow \lfloor (p+r)/2 \rfloor$ 
3   → MERGE-SORT( $A, p, q$ )
4     MERGE-SORT( $A, q+1, r$ )
5     MERGE( $A, p, q, r$ )
    
```

MERGE-SORT(A, p, r)

```

1  $q \leftarrow \lfloor (p+r)/2 \rfloor$ 
2 → MERGE-SORT( $A, p, q$ )
3   MERGE-SORT( $A, q+1, r$ )
4   MERGE( $A, p, q, r$ )
    
```

→ MERGE-SORT(A, p, r)

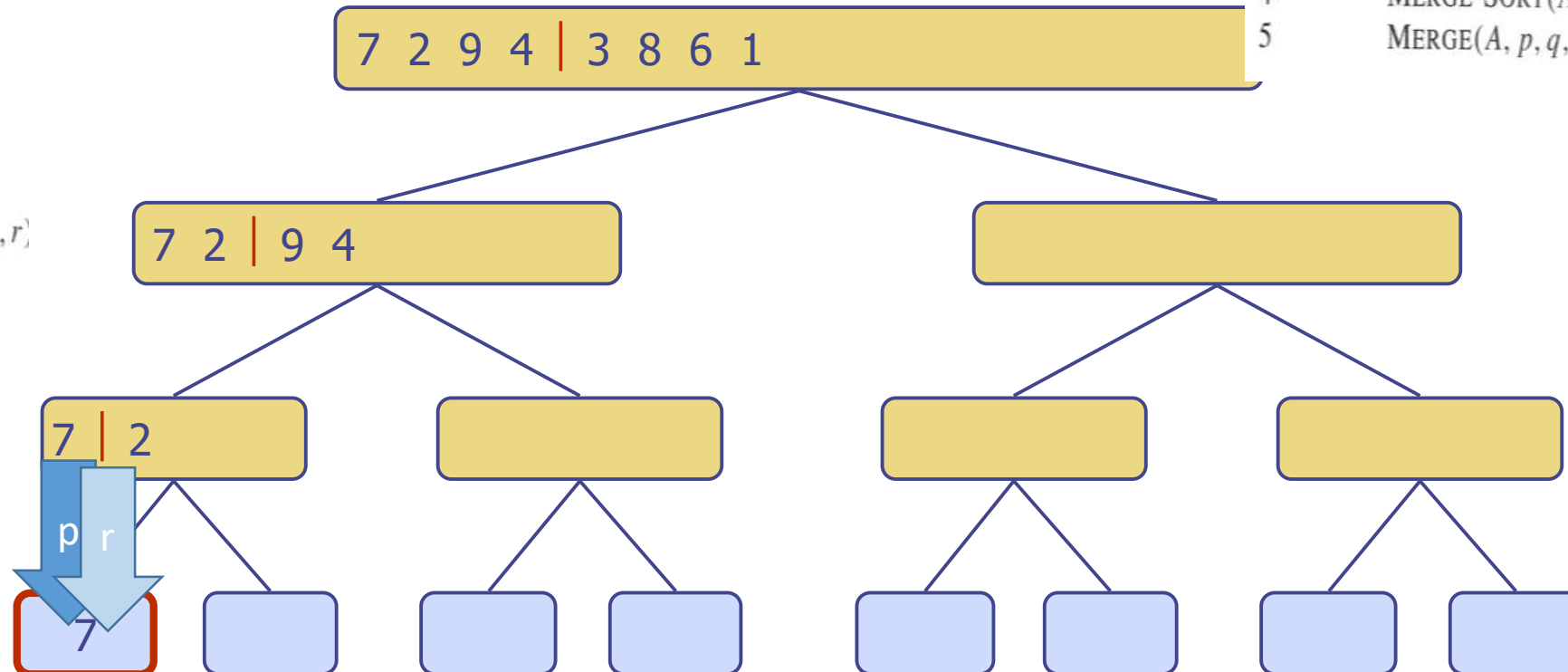
```

1 if  $p < r$ 
2   then  $q \leftarrow \lfloor (p+r)/2 \rfloor$ 
3   MERGE-SORT( $A, p, q$ )
4   MERGE-SORT( $A, q+1, r$ )
    
```

MERGE-SORT(A, p, r)

```

1 if  $p < r$ 
2   then  $q \leftarrow \lfloor (p+r)/2 \rfloor$ 
3   → MERGE-SORT( $A, p, q$ )
4     MERGE-SORT( $A, q+1, r$ )
5     MERGE( $A, p, q, r$ )
    
```



Merge Sort Simulation

MERGE-SORT(A, p, r)

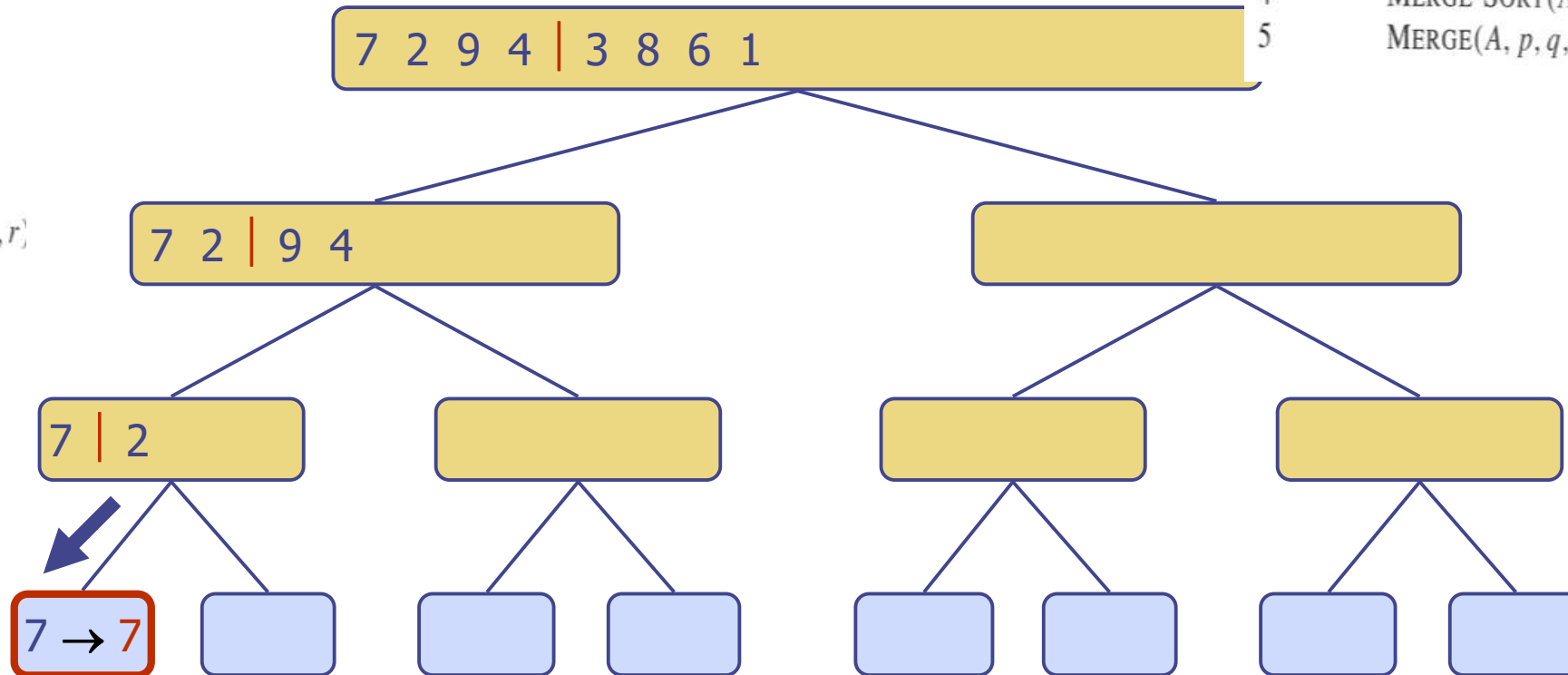
```
1 if  $p < r$ 
2   then  $q \leftarrow \lfloor (p+r)/2 \rfloor$ 
3   → MERGE-SORT( $A, p, q$ )
4     MERGE-SORT( $A, q+1, r$ )
5     MERGE( $A, p, q, r$ )
```

MERGE-SORT(A, p, r)

```
1  $q \leftarrow \lfloor (p+r)/2 \rfloor$ 
2   → MERGE-SORT( $A, p, q$ )
3   MERGE-SORT( $A, q+1, r$ )
4   MERGE( $A, p, q, r$ )
```

MERGE-SORT(A, p, r)

```
1 if  $p < r$ 
2   then  $q \leftarrow \lfloor (p+r)/2 \rfloor$ 
3   → MERGE-SORT( $A, p, q$ )
4     MERGE-SORT( $A, q+1, r$ )
5     MERGE( $A, p, q, r$ )
```




```

1  if  $p < r$ 
2    then  $q \leftarrow \lfloor (p+r)/2 \rfloor$ 
3     MERGE-SORT( $A, p, q$ )
4      MERGE-SORT( $A, q+1, r$ )
5      MERGE( $A, p, q, r$ )

```



Merge Sort Simulation

```

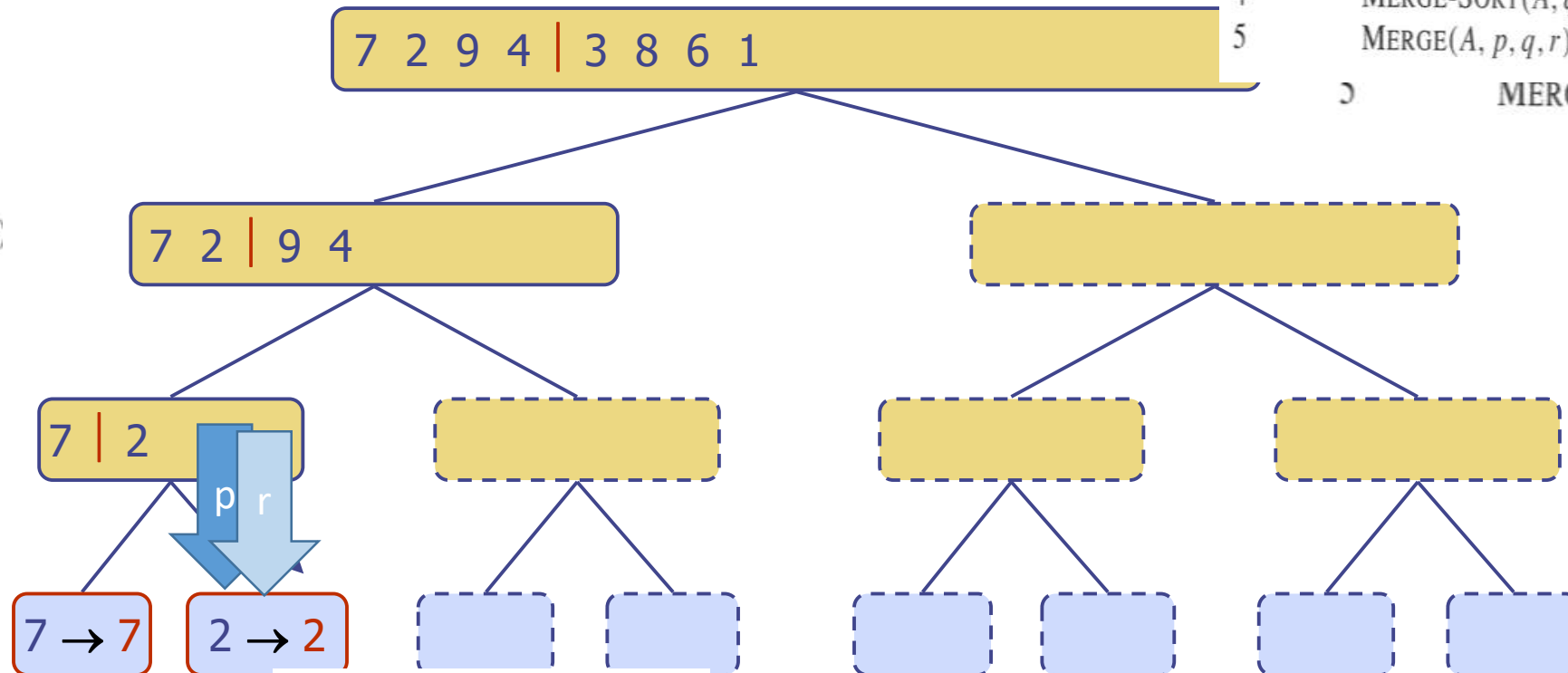
MERGE-SORT(A, p, r)
1  if p < r
2    then q ← ⌊(p+r)/2⌋
3    → MERGE-SORT(A, p, q)
4    MERGE-SORT(A, q+1, r)
5    MERGE(A, p, q, r)
    
```

```

MERGE-SORT(A, p, r)
1  if p < r
2    then q ← ⌊(p+r)/2⌋
3    → MERGE-SORT(A, p, q)
4    MERGE-SORT(A, q+1, r)
5    MERGE(A, p, q, r)
    
```

```

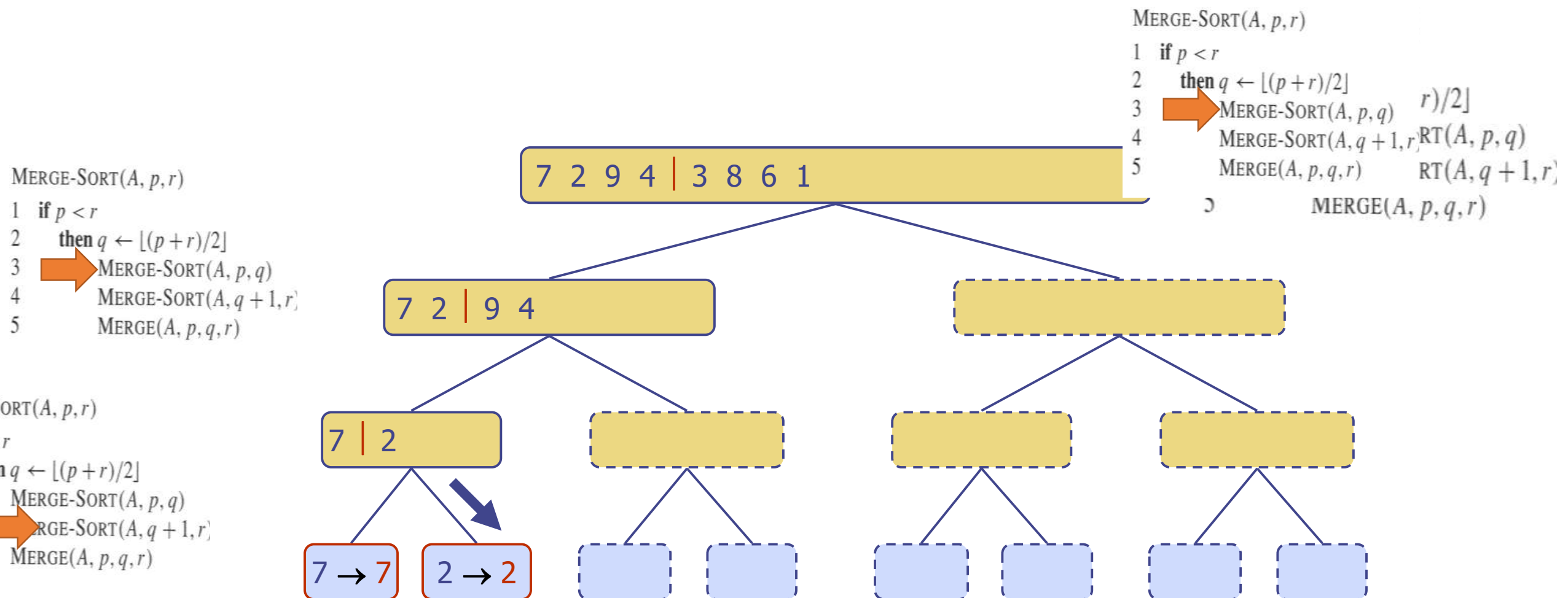
MERGE-SORT(A, p, r)
1  if p < r
2    then q ← ⌊(p+r)/2⌋
3    → MERGE-SORT(A, p, q)
4    MERGE-SORT(A, q+1, r)
5    MERGE(A, p, q, r)
    
```



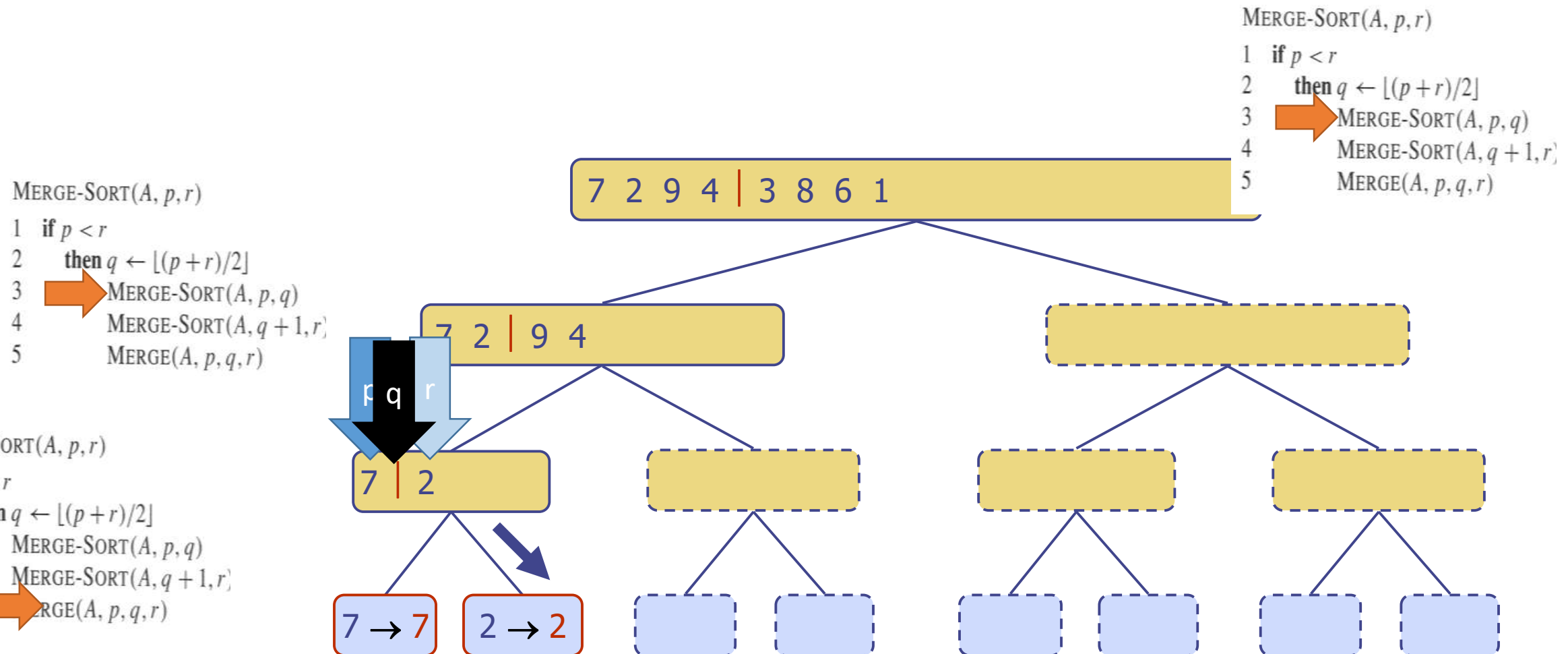
```

MERGE-SORT(A, p, r)
1  if p < r
2    then q ← ⌊(p+r)/2⌋
3    MERGE-SORT(A, p, q)
4    MERGE-SORT(A, q+1, r)
5    MERGE(A, p, q, r)
    
```

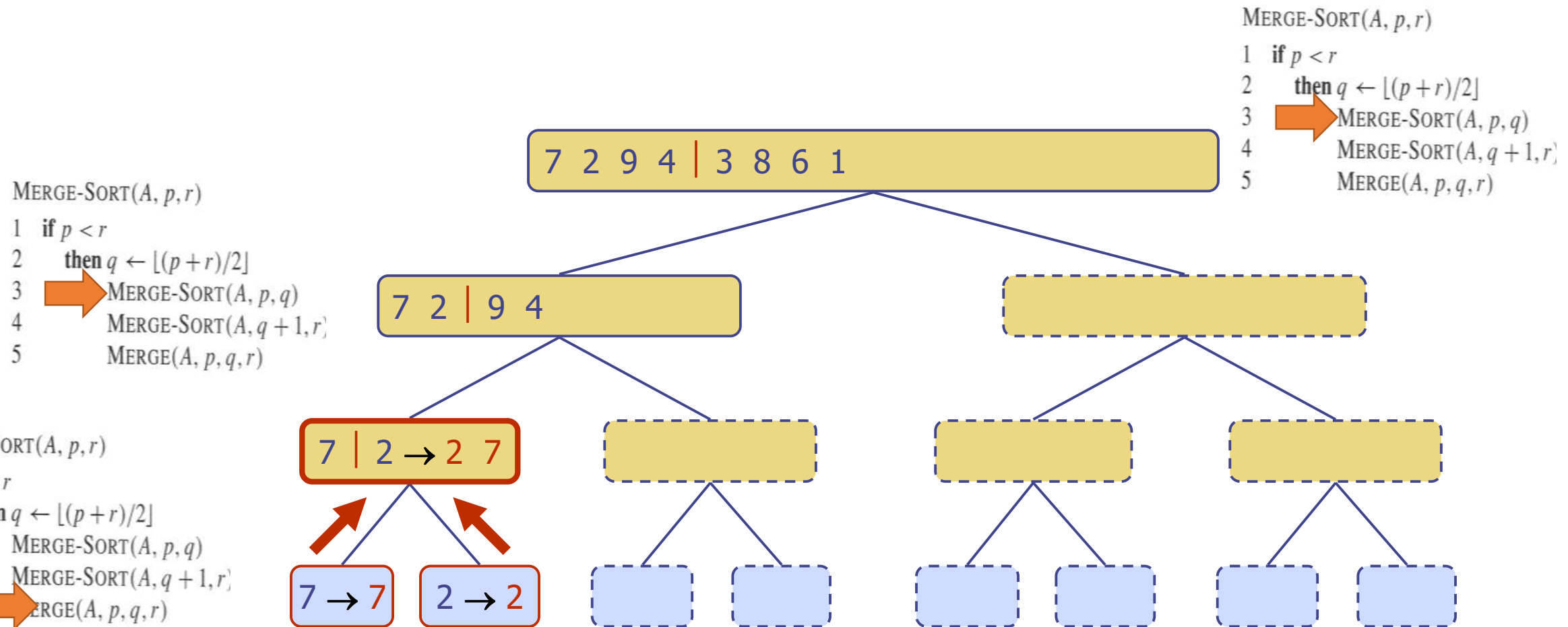
Merge Sort Simulation



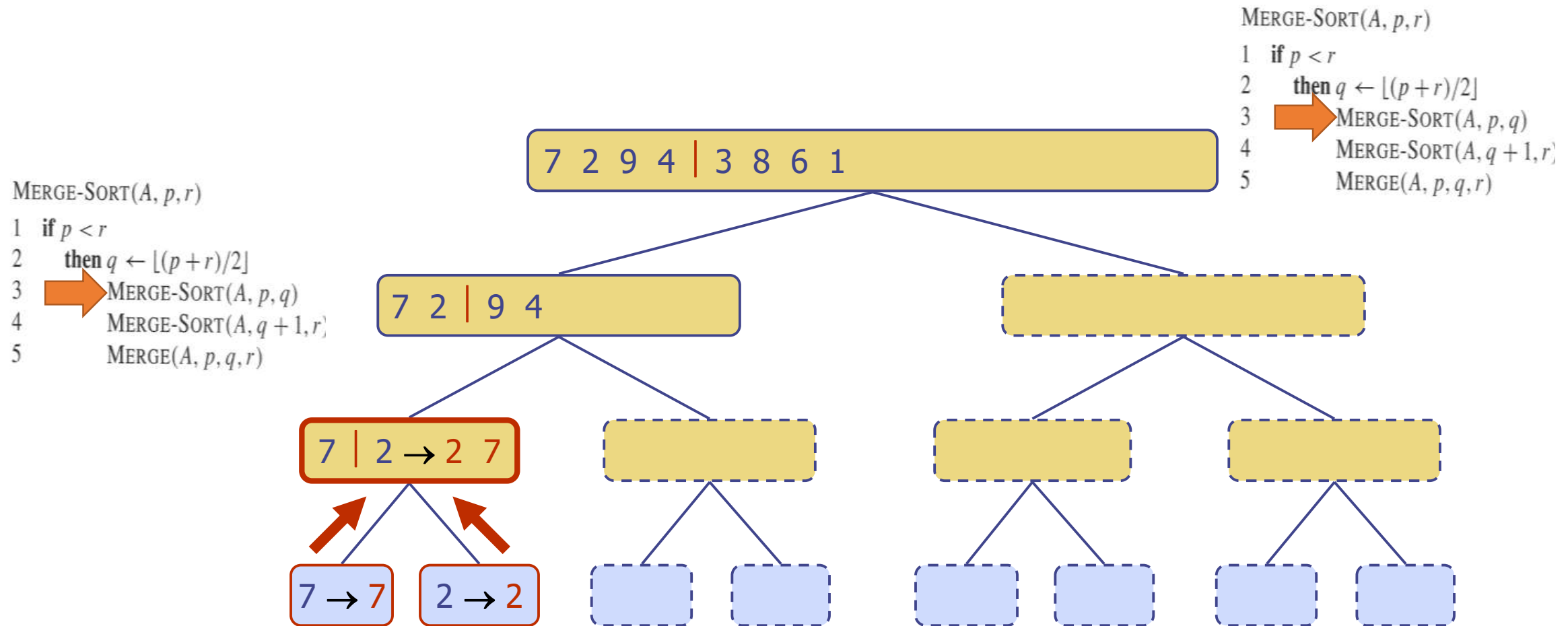
Merge Sort Simulation



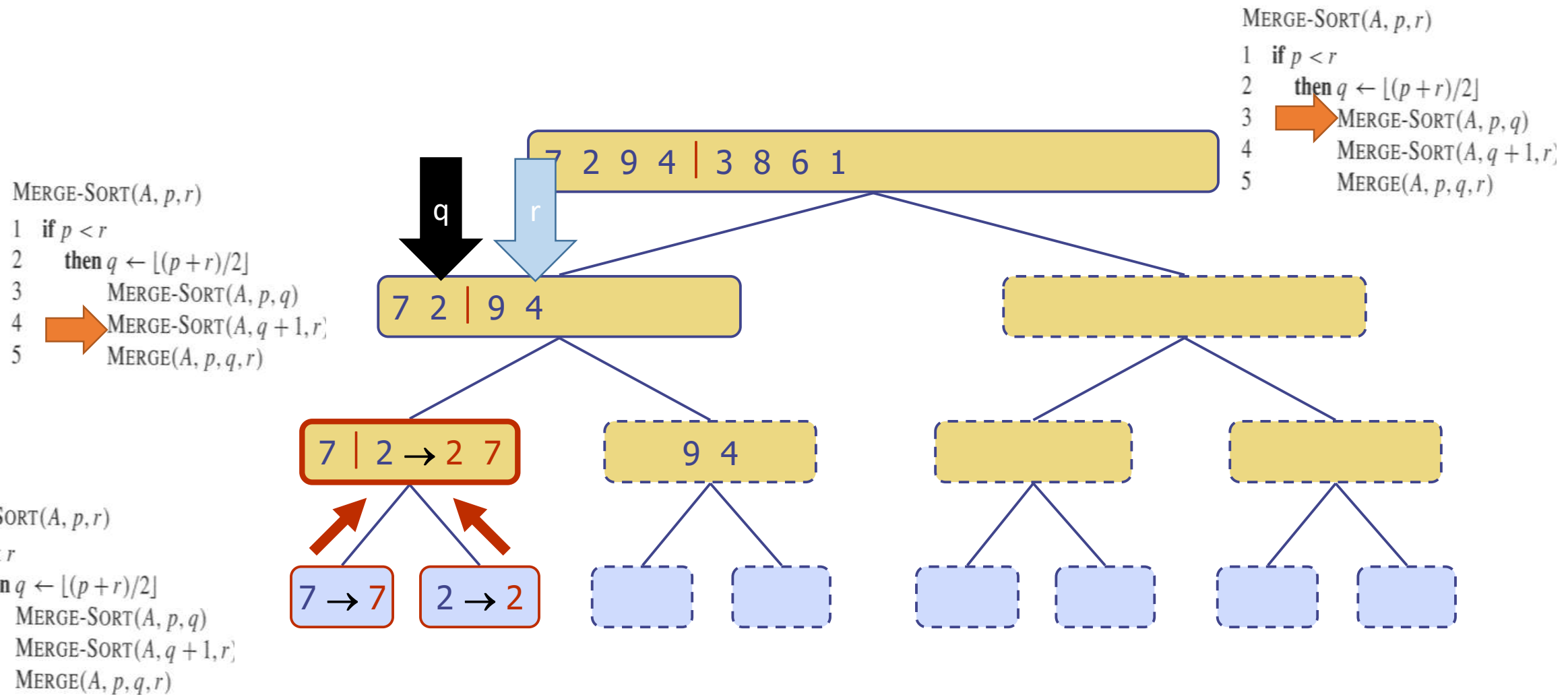
Merge Sort Simulation



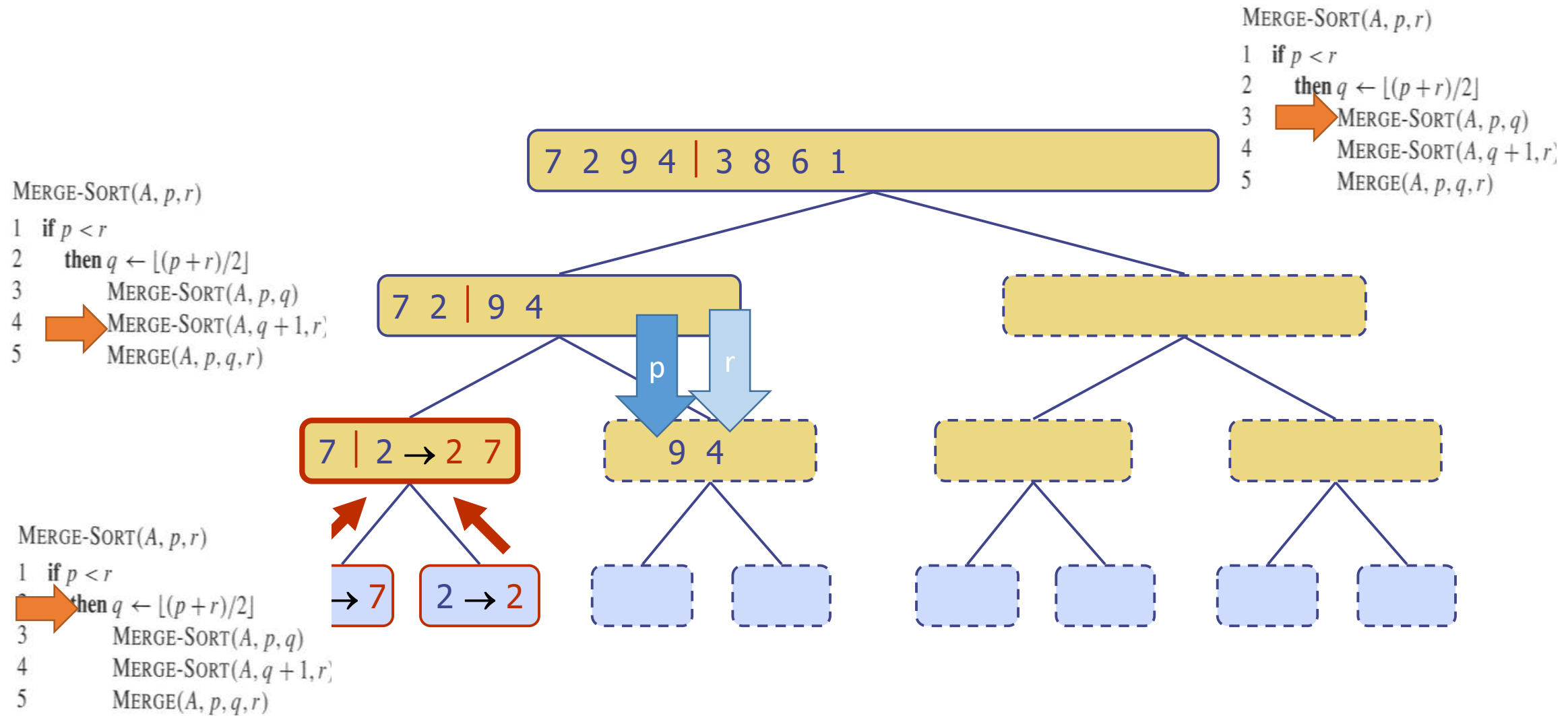
Merge Sort Simulation



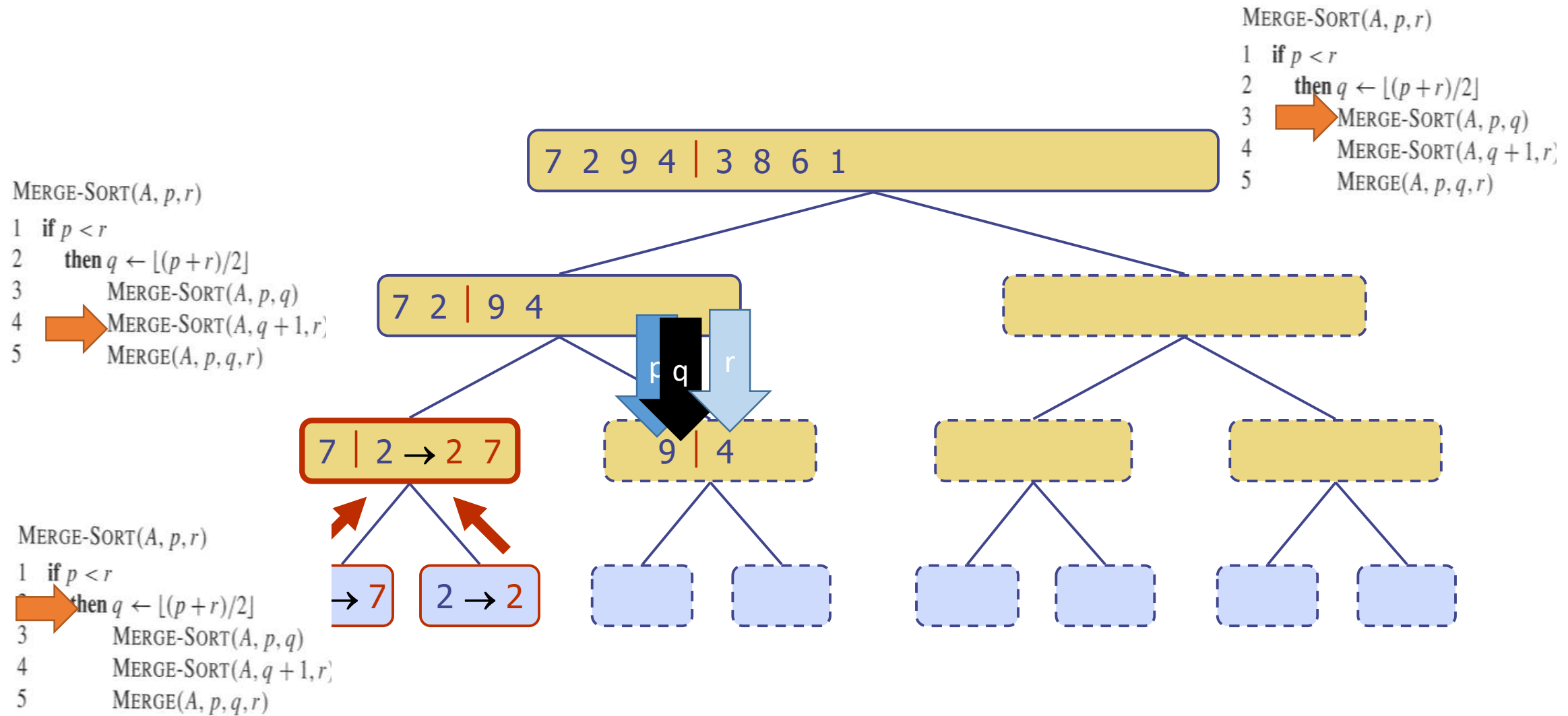
Merge Sort Simulation



Merge Sort Simulation



Merge Sort Simulation



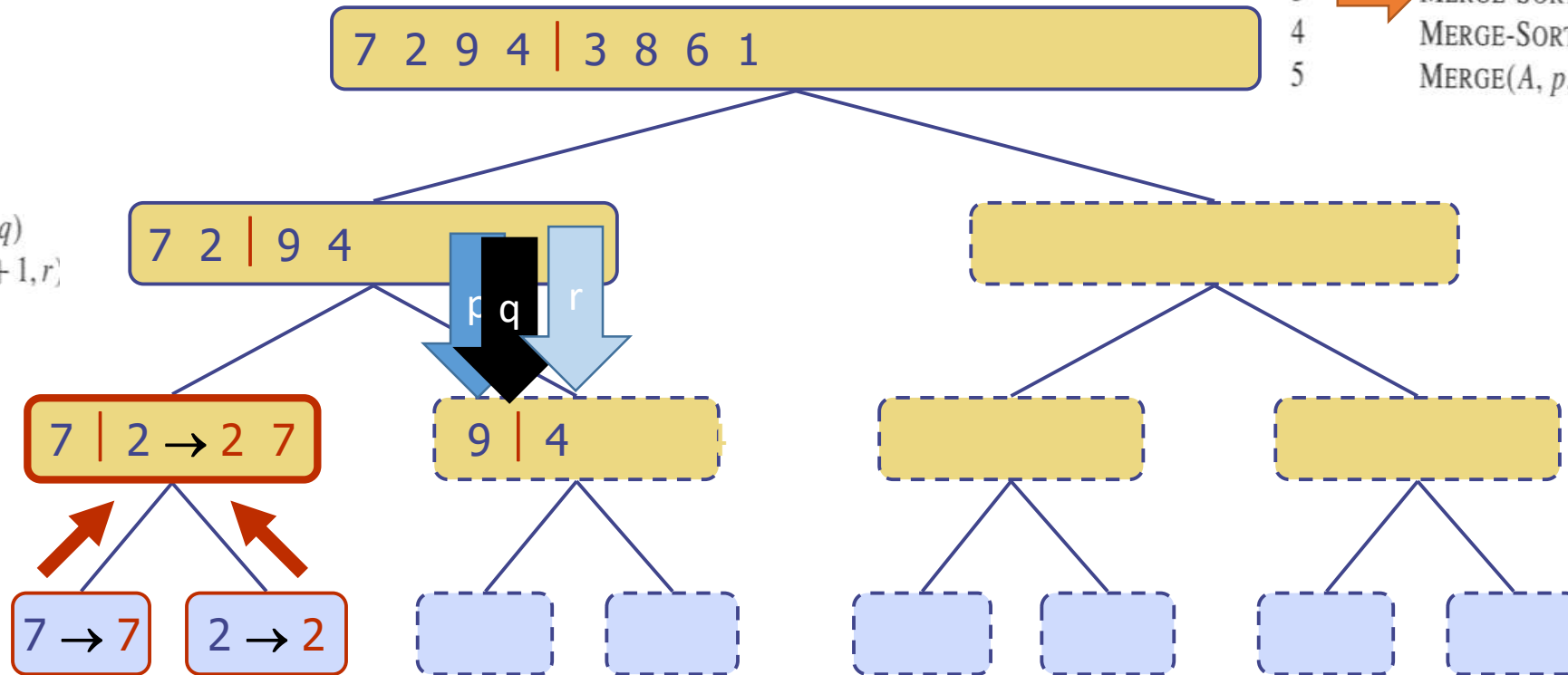
Merge Sort Simulation

```

MERGE-SORT(A, p, r)
1  if p < r
2    then q ← ⌊(p+r)/2⌋
3    MERGE-SORT(A, p, q)
4    MERGE-SORT(A, q+1, r)
5    MERGE(A, p, q, r)
    
```

```

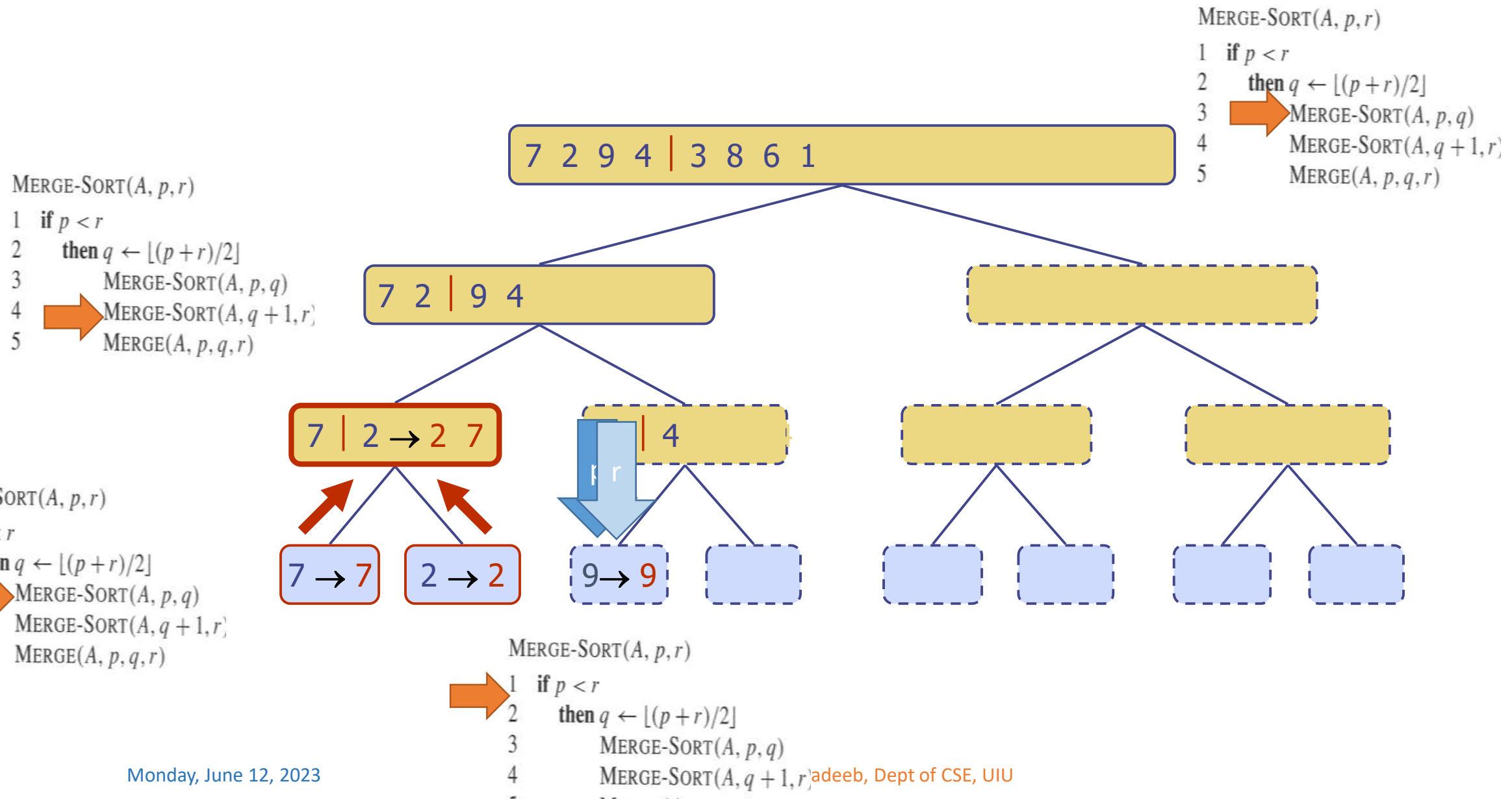
MERGE-SORT(A, p, r)
1  if p < r
2    then q ← ⌊(p+r)/2⌋
3    MERGE-SORT(A, p, q)
4    MERGE-SORT(A, q+1, r)
5    MERGE(A, p, q, r)
    
```



```

MERGE-SORT(A, p, r)
1  if p < r
2    then q ← ⌊(p+r)/2⌋
3    MERGE-SORT(A, p, q)
4    MERGE-SORT(A, q+1, r)
5    MERGE(A, p, q, r)
    
```

Merge Sort Simulation



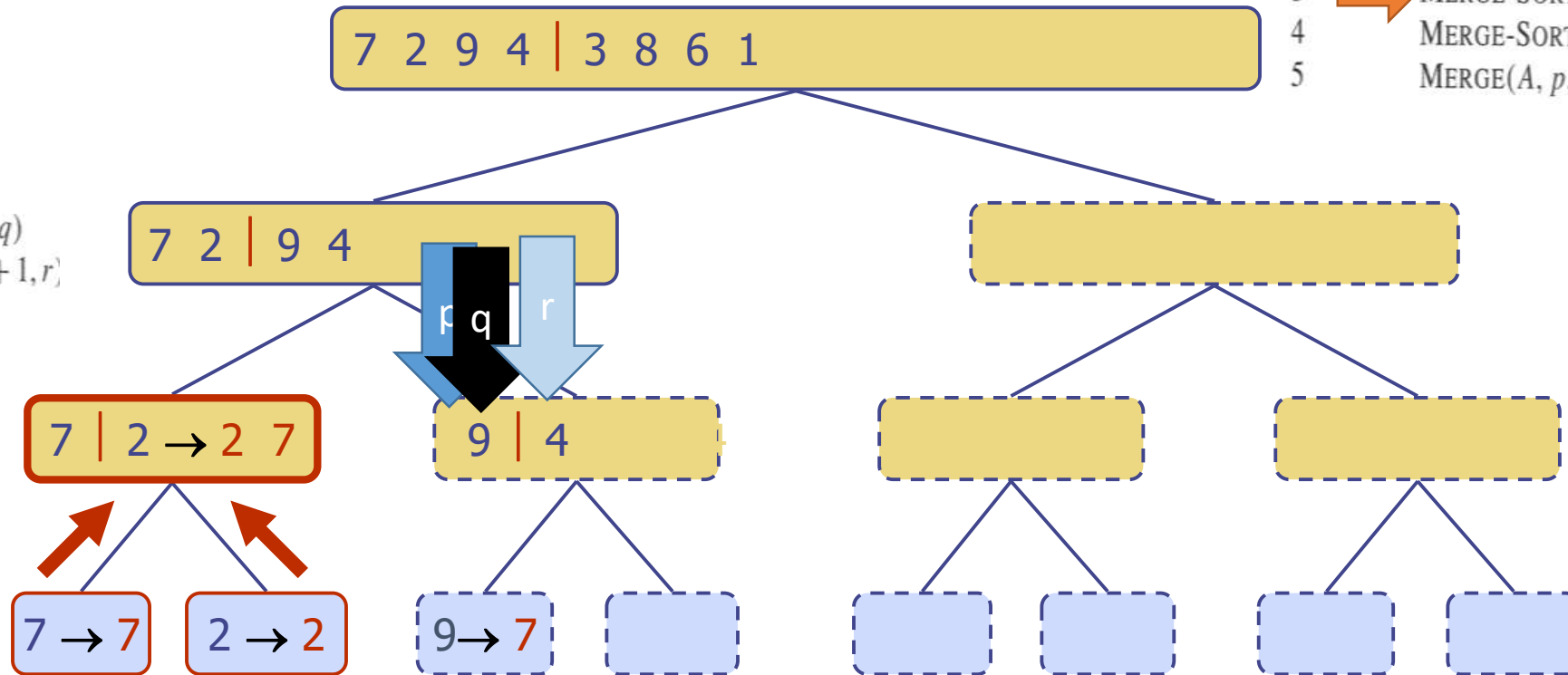
Merge Sort Simulation

```

MERGE-SORT(A, p, r)
1  if p < r
2    then q ← ⌊(p+r)/2⌋
3    MERGE-SORT(A, p, q)
4    MERGE-SORT(A, q+1, r)
5    MERGE(A, p, q, r)
    
```

```

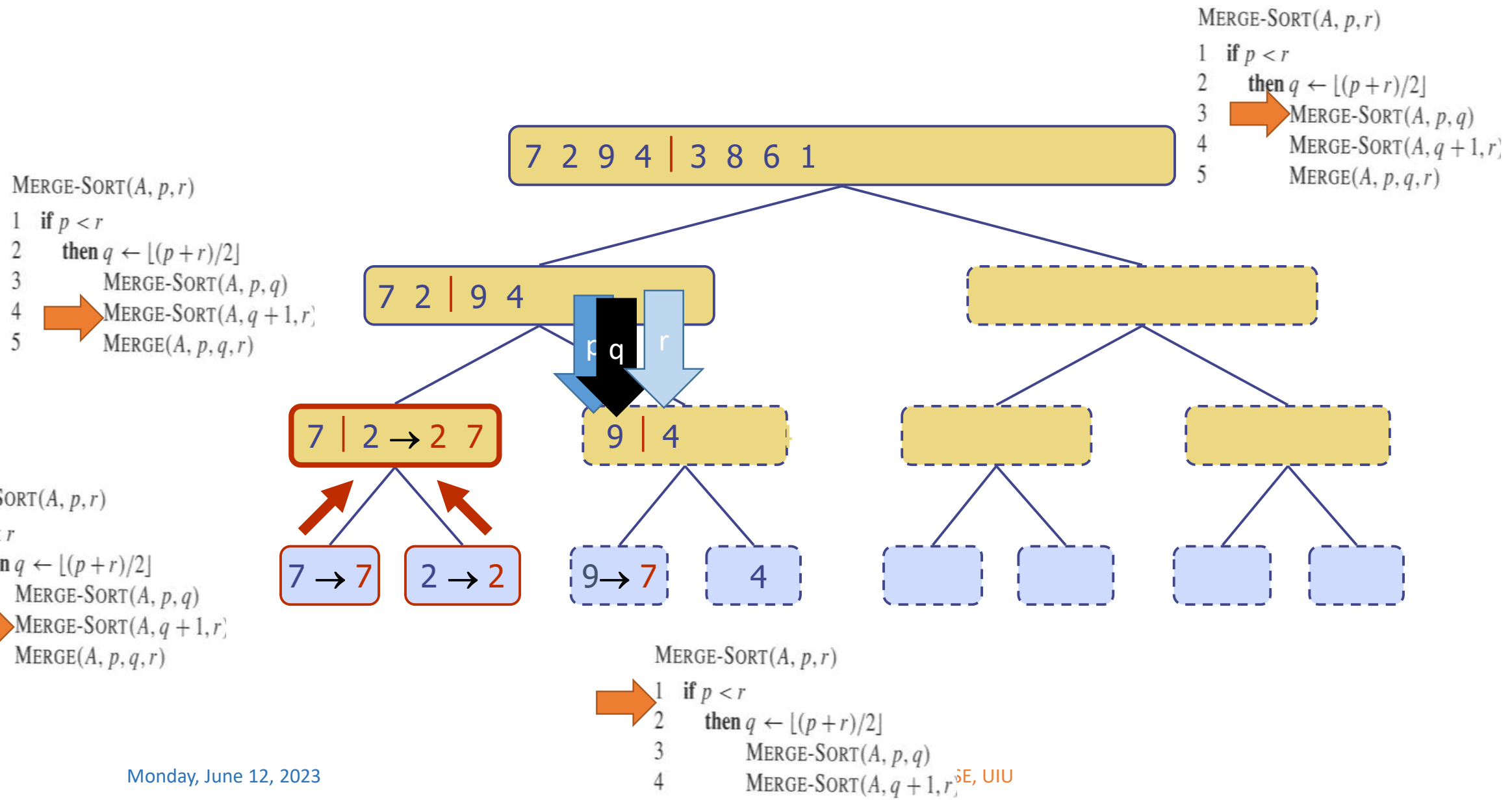
MERGE-SORT(A, p, r)
1  if p < r
2    then q ← ⌊(p+r)/2⌋
3    MERGE-SORT(A, p, q)
4    MERGE-SORT(A, q+1, r)
5    MERGE(A, p, q, r)
    
```



```

MERGE-SORT(A, p, r)
1  if p < r
2    then q ← ⌊(p+r)/2⌋
3    MERGE-SORT(A, p, q)
4    MERGE-SORT(A, q+1, r)
5    MERGE(A, p, q, r)
    
```

Merge Sort Simulation



Merge Sort Simulation

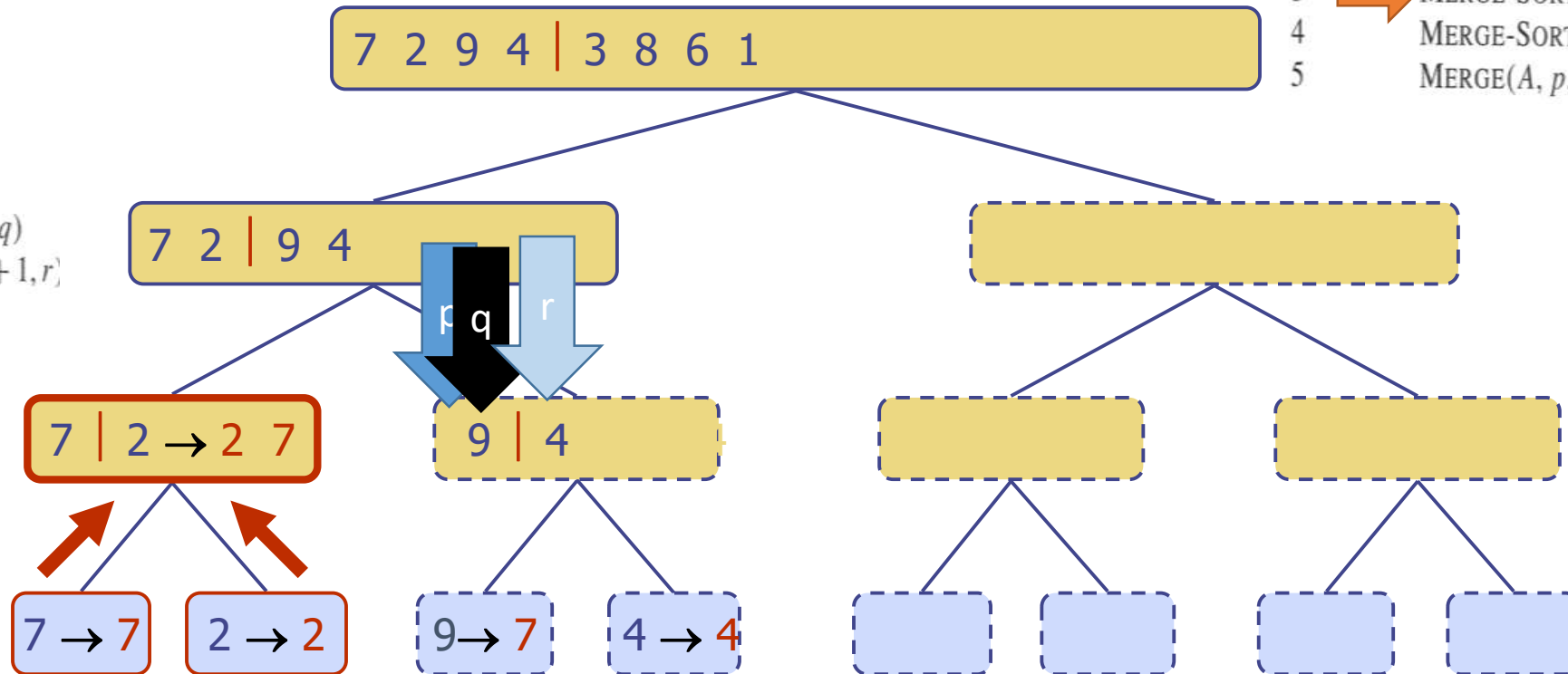
```

MERGE-SORT(A, p, r)
1  if p < r
2    then q ← ⌊(p+r)/2⌋
3    MERGE-SORT(A, p, q)
4    MERGE-SORT(A, q+1, r)
5    MERGE(A, p, q, r)
    
```

MERGE-SORT(A, p, r)

```

1  if p < r
2    then q ← ⌊(p+r)/2⌋
3    MERGE-SORT(A, p, q)
4    MERGE-SORT(A, q+1, r)
5    MERGE(A, p, q, r)
    
```



```

MERGE-SORT(A, p, r)
1  if p < r
2    then q ← ⌊(p+r)/2⌋
3    MERGE-SORT(A, p, q)
4    MERGE-SORT(A, q+1, r)
5    MERGE(A, p, q, r)
    
```

Merge Sort Simulation

```

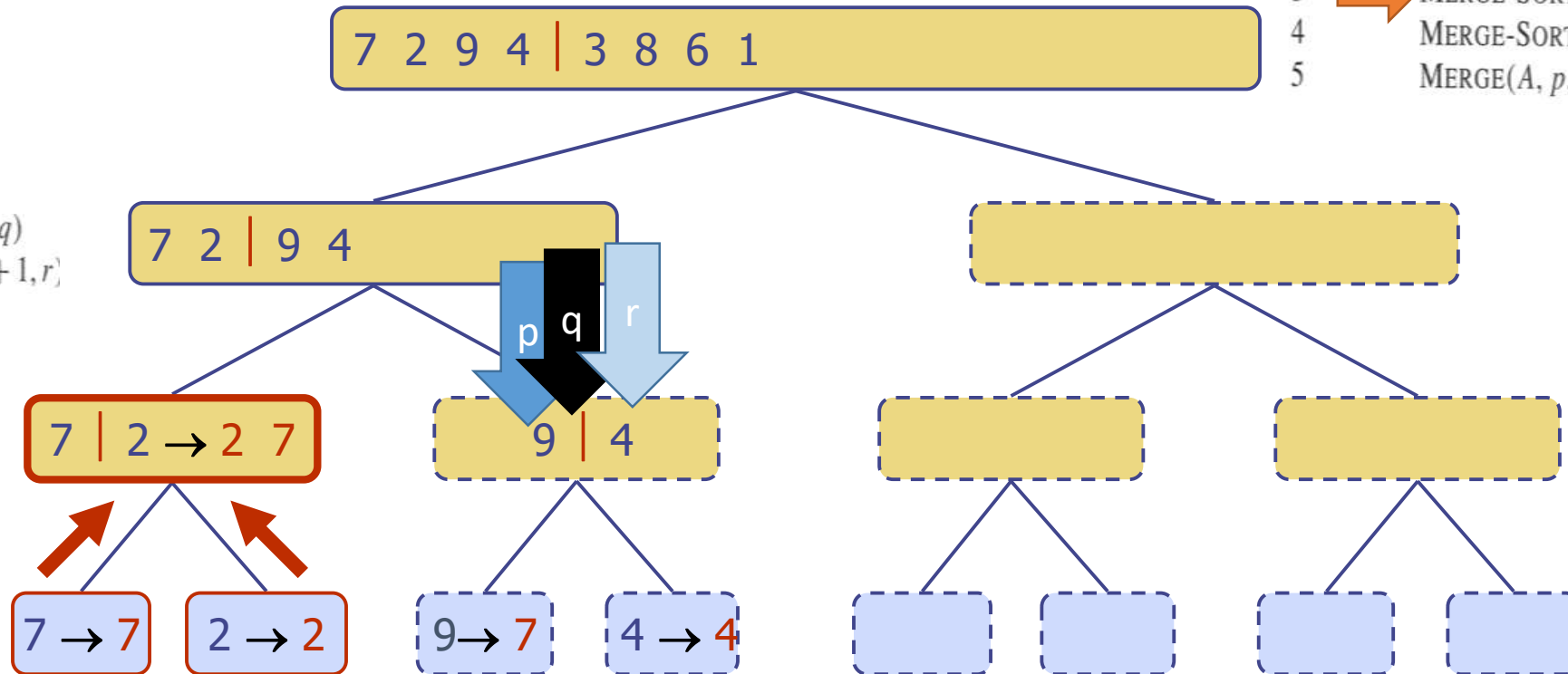
MERGE-SORT(A, p, r)
1  if p < r
2    then q ← ⌊(p+r)/2⌋
3    MERGE-SORT(A, p, q)
4    MERGE-SORT(A, q+1, r)
5    MERGE(A, p, q, r)
    
```

```

MERGE-SORT(A, p, r)
1  if p < r
2    then q ← ⌊(p+r)/2⌋
3    MERGE-SORT(A, p, q)
4    MERGE-SORT(A, q+1, r)
5    MERGE(A, p, q, r)
    
```

```

MERGE-SORT(A, p, r)
1  if p < r
2    then q ← ⌊(p+r)/2⌋
3    MERGE-SORT(A, p, q)
4    MERGE-SORT(A, q+1, r)
5    MERGE(A, p, q, r)
    
```



Merge Sort Simulation

```

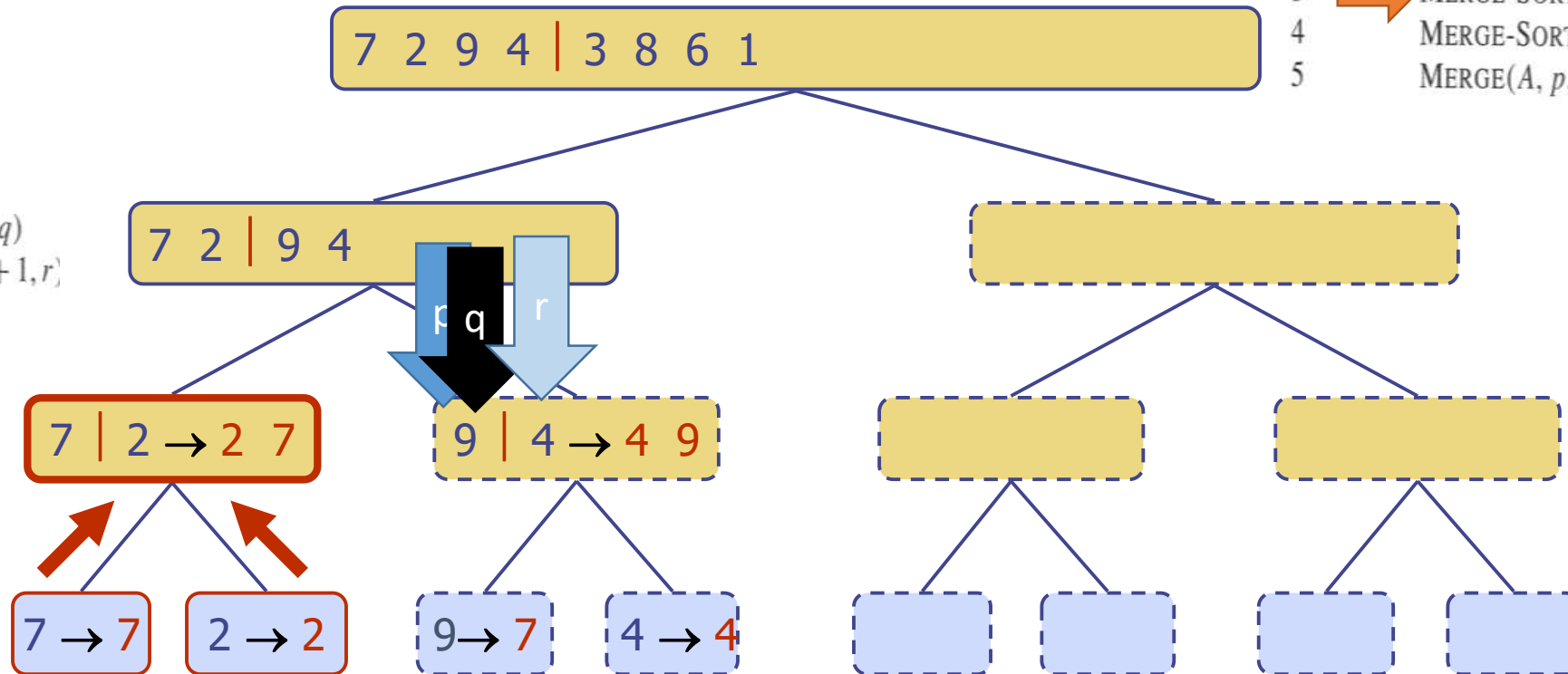
MERGE-SORT(A, p, r)
1  if p < r
2    then q ← ⌊(p+r)/2⌋
3    MERGE-SORT(A, p, q)
4    → MERGE-SORT(A, q+1, r)
5    MERGE(A, p, q, r)
    
```

```

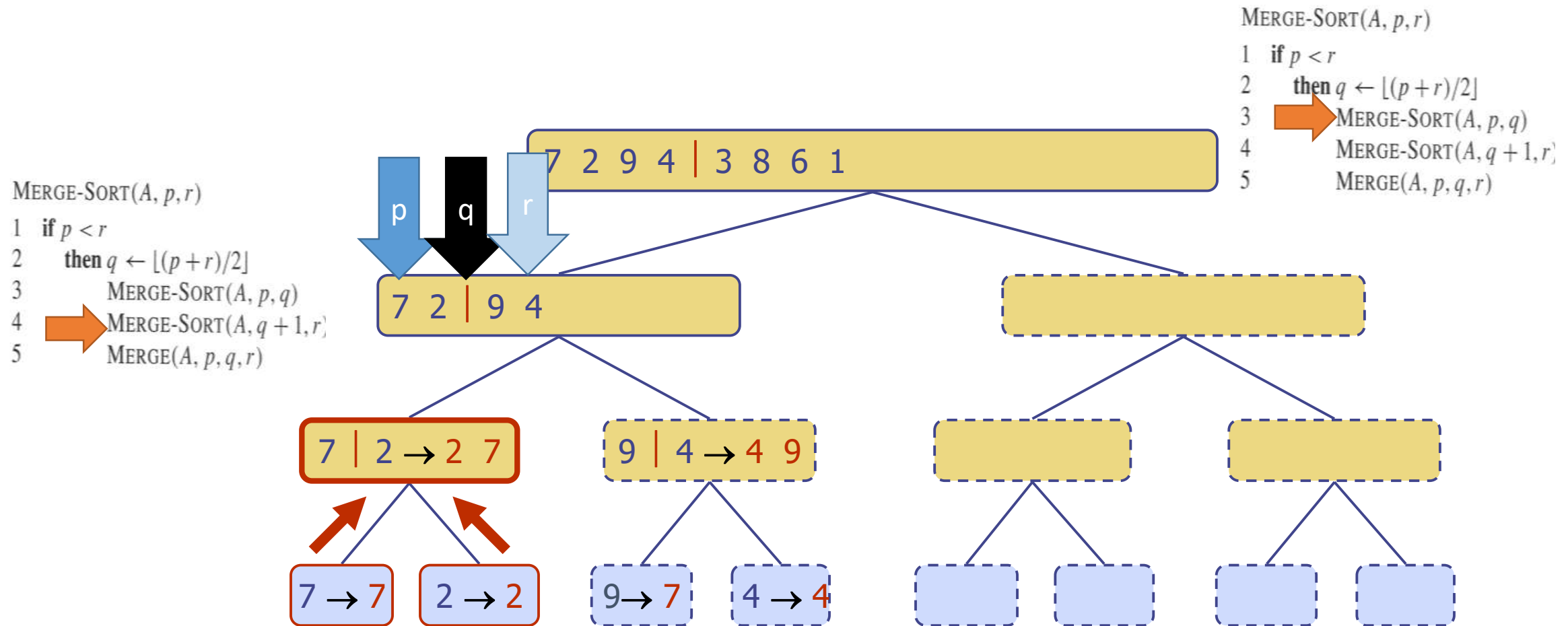
MERGE-SORT(A, p, r)
1  if p < r
2    then q ← ⌊(p+r)/2⌋
3    → MERGE-SORT(A, p, q)
4    MERGE-SORT(A, q+1, r)
5    MERGE(A, p, q, r)
    
```

```

MERGE-SORT(A, p, r)
1  if p < r
2    then q ← ⌊(p+r)/2⌋
3    MERGE-SORT(A, p, q)
4    MERGE-SORT(A, q+1, r)
5    → MERGE(A, p, q, r)
    
```



Merge Sort Simulation



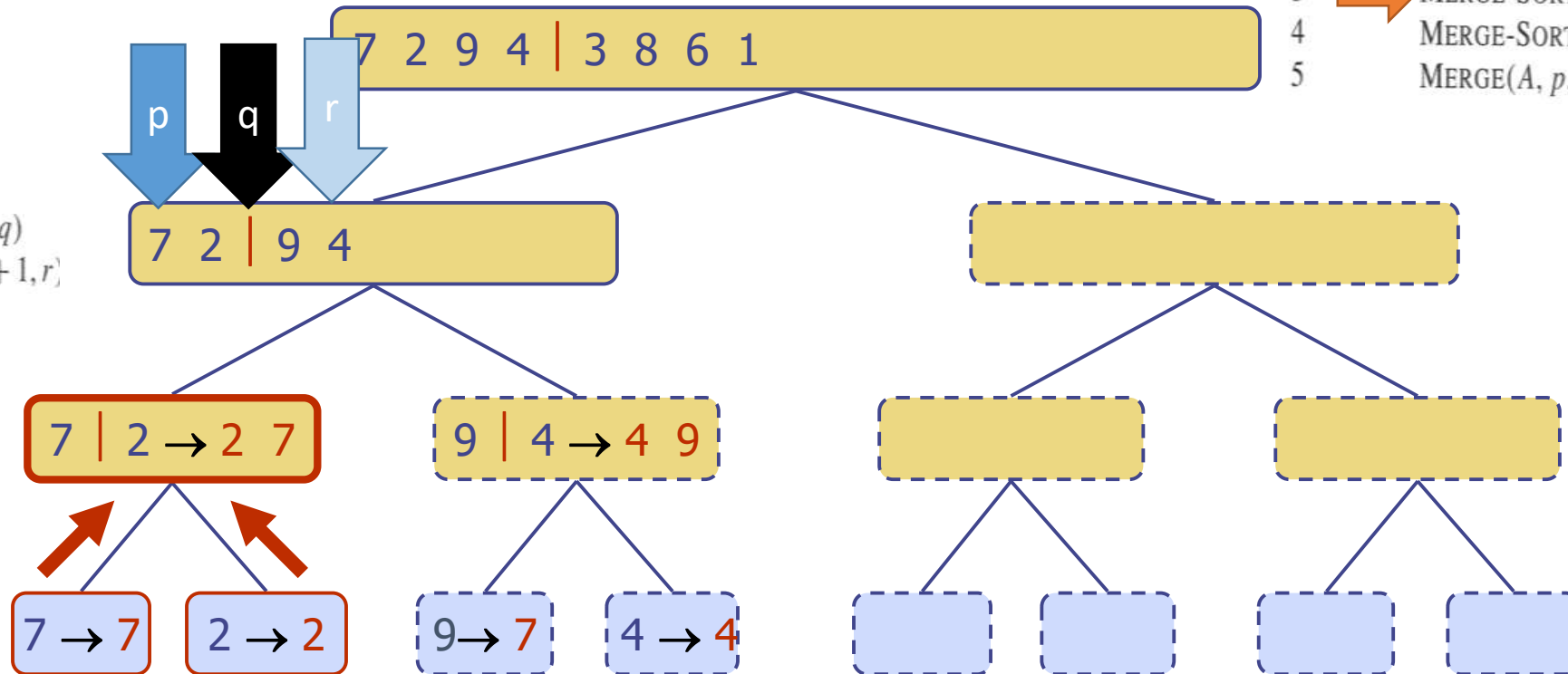
Merge Sort Simulation

MERGE-SORT(A, p, r)

- 1 if $p < r$
- 2 then $q \leftarrow \lfloor (p+r)/2 \rfloor$
- 3 MERGE-SORT(A, p, q)
- 4 MERGE-SORT($A, q+1, r$)
- 5 MERGE(A, p, q, r)

MERGE-SORT(A, p, r)

- 1 if $p < r$
- 2 then $q \leftarrow \lfloor (p+r)/2 \rfloor$
- 3 MERGE-SORT(A, p, q)
- 4 MERGE-SORT($A, q+1, r$)
- 5 MERGE(A, p, q, r)



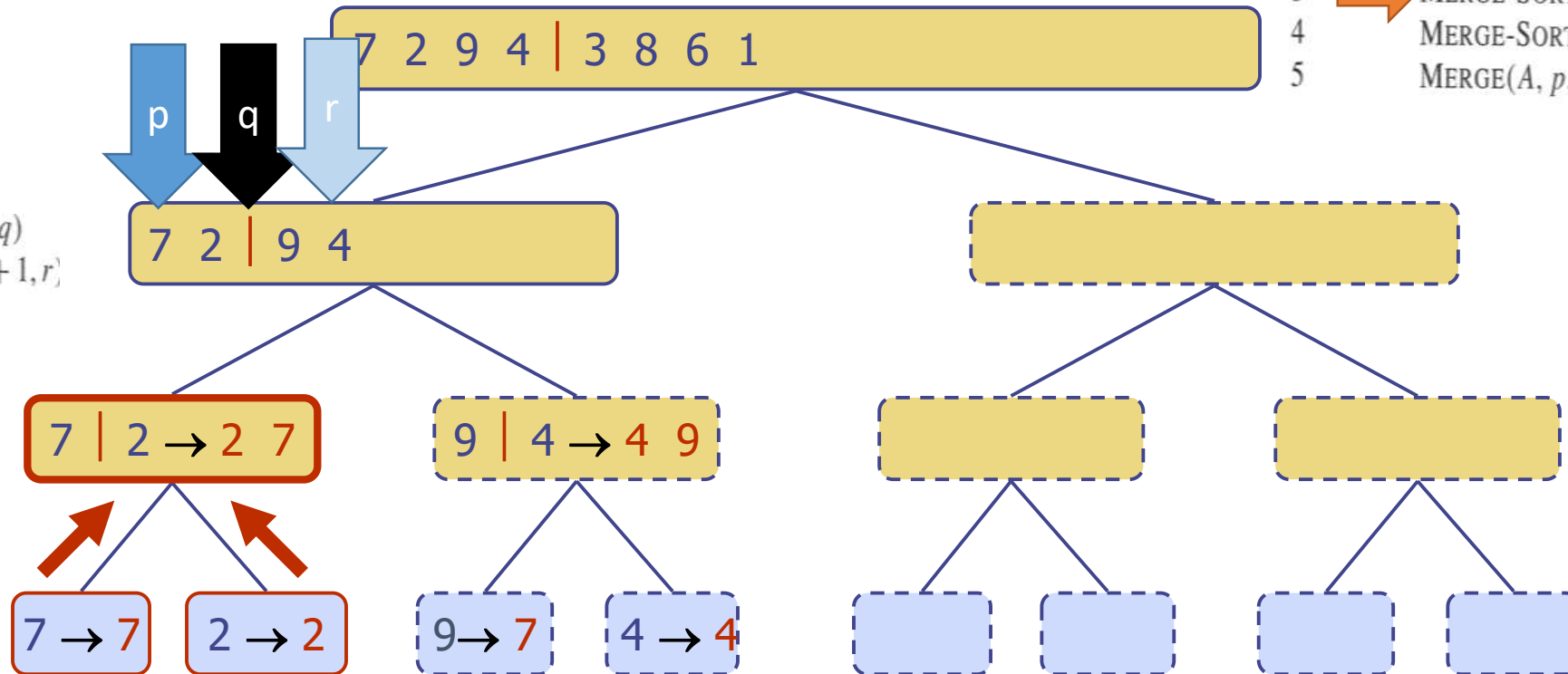
Merge Sort Simulation

MERGE-SORT(A, p, r)

- 1 if $p < r$
- 2 then $q \leftarrow \lfloor (p+r)/2 \rfloor$
- 3 MERGE-SORT(A, p, q)
- 4 MERGE-SORT($A, q+1, r$)
- 5 MERGE(A, p, q, r)

MERGE-SORT(A, p, r)

- 1 if $p < r$
- 2 then $q \leftarrow \lfloor (p+r)/2 \rfloor$
- 3 MERGE-SORT(A, p, q)
- 4 MERGE-SORT($A, q+1, r$)
- 5 MERGE(A, p, q, r)



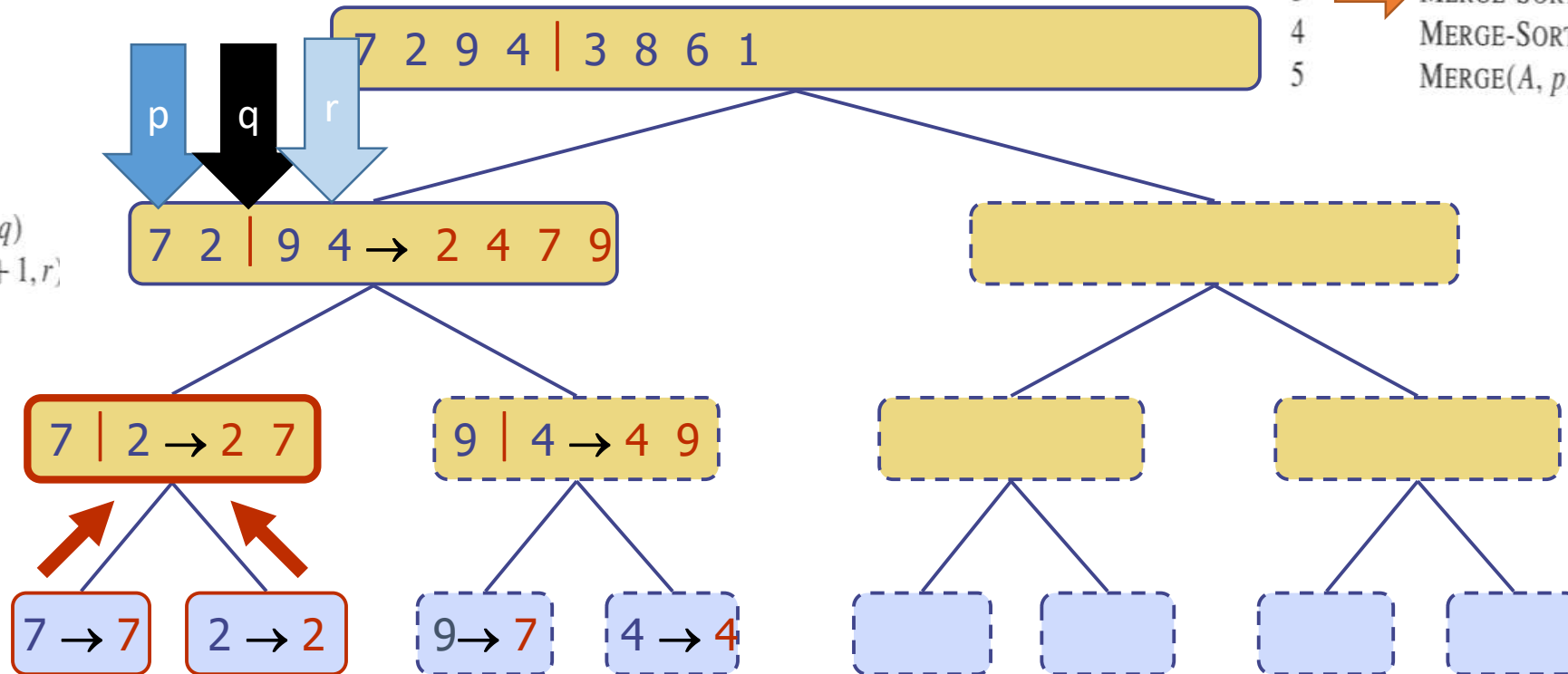
Merge Sort Simulation

MERGE-SORT(A, p, r)

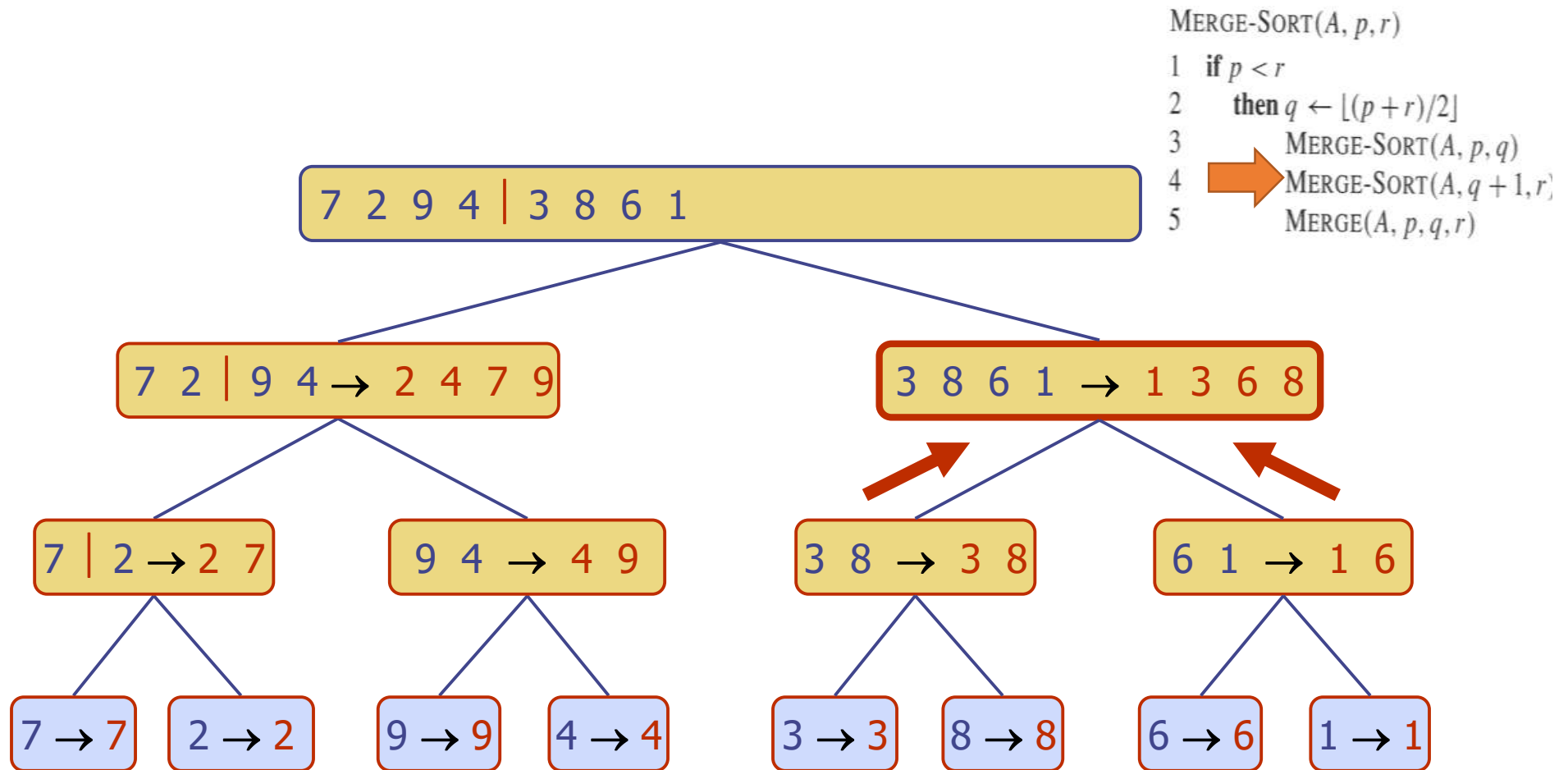
- 1 if $p < r$
- 2 then $q \leftarrow \lfloor (p+r)/2 \rfloor$
- 3 MERGE-SORT(A, p, q)
- 4 MERGE-SORT($A, q+1, r$)
- 5 MERGE(A, p, q, r)

MERGE-SORT(A, p, r)

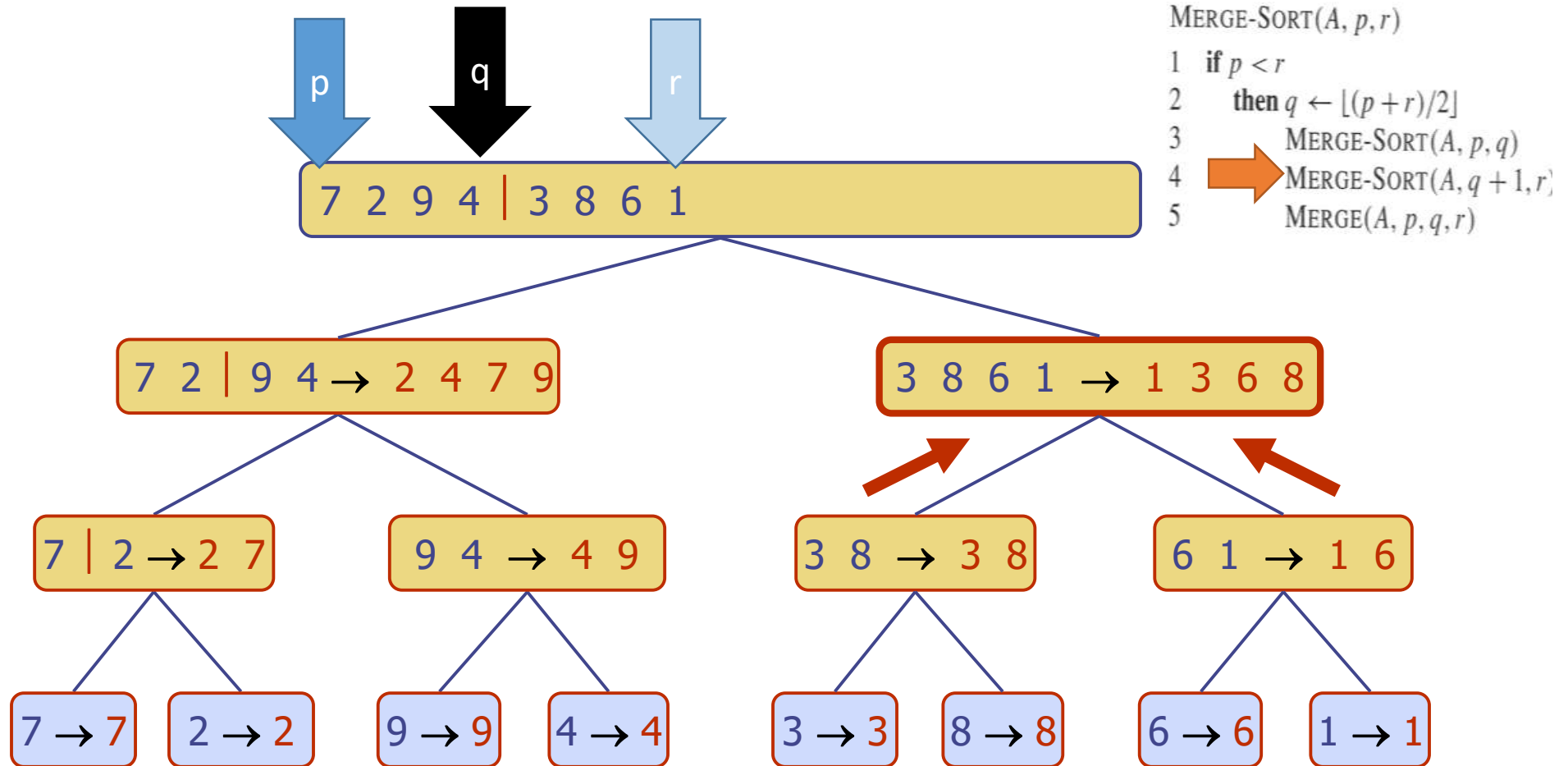
- 1 if $p < r$
- 2 then $q \leftarrow \lfloor (p+r)/2 \rfloor$
- 3 MERGE-SORT(A, p, q)
- 4 MERGE-SORT($A, q+1, r$)
- 5 MERGE(A, p, q, r)



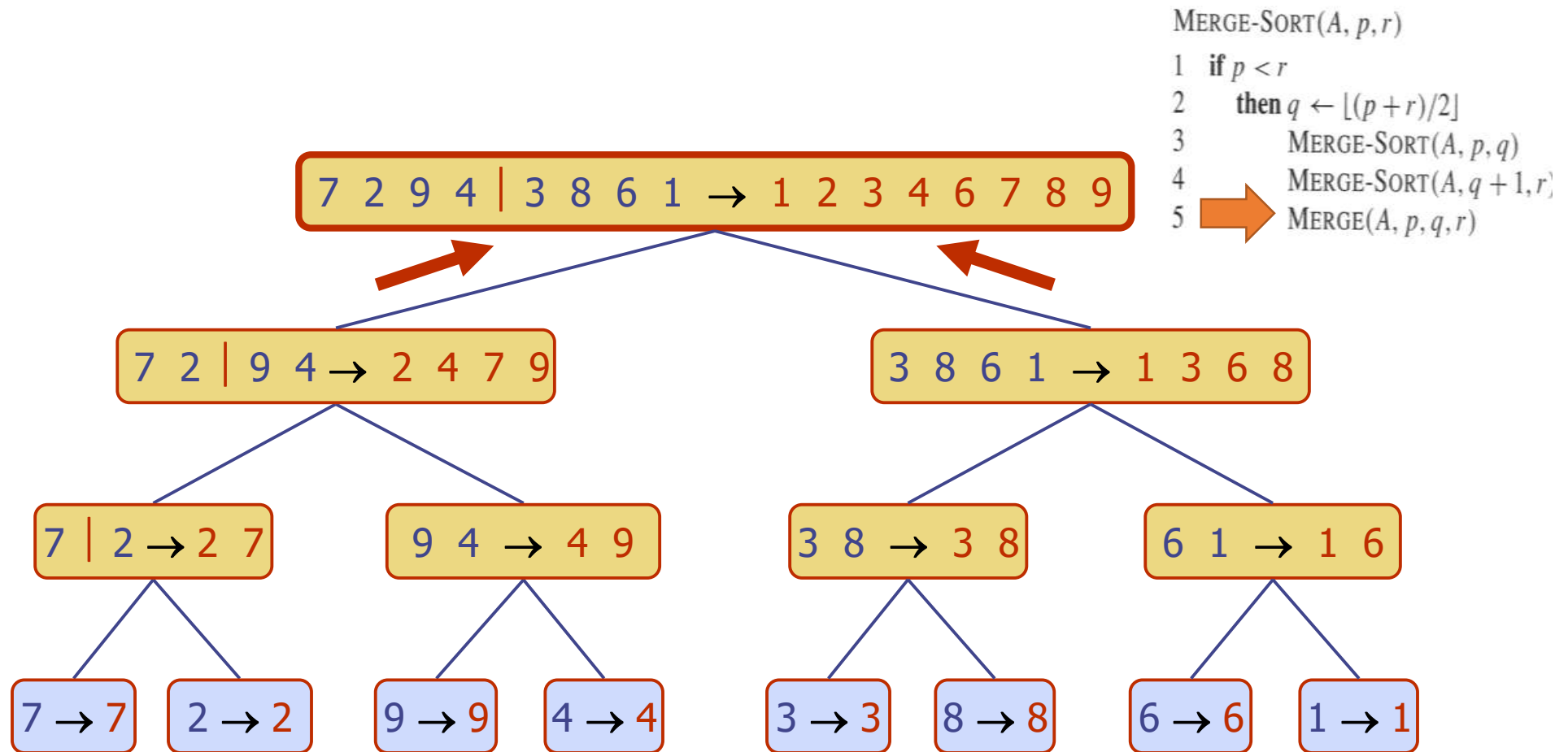
Merge Sort Simulation



Merge Sort Simulation



Merge Sort Simulation



Merge Sort: Running Time

The recurrence for the worst-case running time $T(n)$ is

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1 \\ 2T(n/2) + \Theta(n) & \text{if } n > 1 \end{cases}$$

equivalently

$$T(n) = \begin{cases} b & \text{if } n = 1 \\ 2T(n/2) + bn & \text{if } n > 1 \end{cases}$$

Solve this recurrence by

- (1) iteratively expansion
- (2) using the recursion tree

Merge Sort: Running Time

$$\begin{aligned}T(n) &= 2T(n/2) + bn \\&= 2(2T(n/2^2)) + b(n/2) + bn \\&= 2^2T(n/2^2) + 2bn \\&= 2^3T(n/2^3) + 3bn \\&= 2^4T(n/2^4) + 4bn \\&= \dots \\&= 2^iT(n/2^i) + ibn\end{aligned}$$

◆ Note that base, $T(n) = b$, case occurs when $2^i = n$.
That is, $i = \log n$.

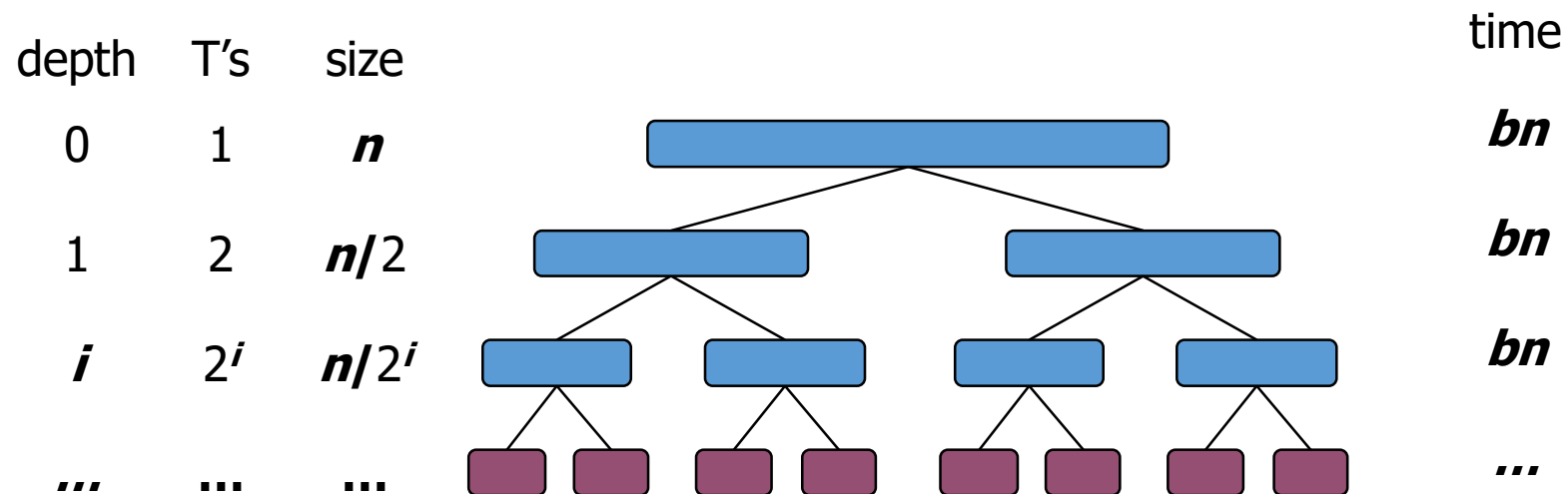
◆ So, $T(n) = bn + bn \log n$

◆ Thus, $T(n)$ is $O(n \log n)$.

Merge Sort: Running Time (Recursion Tree)

- Draw the recursion tree for the recurrence relation and look for a pattern:

$$T(n) = \begin{cases} b & \text{if } n = 1 \\ 2T(n/2) + bn & \text{if } n \geq 2 \end{cases}$$



Total time = $bn + bn \log n$
(last level plus all previous levels)



Thank You